LectureAssist:
An Agentic, Multimodal AI Assistant that Turns Lectures into Adaptive Study Material

By

Iftikhar Ahmed **2121019642**
Salman Shafi **2211386042**
Jarinah Tasnim **2121026642**

Supervised by
Dr. Md Adnan Arefeen
Assistant Professor

Submitted in partial fulfillment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

August 2026

Department of Electrical and Computer Engineering
North South University

LectureAssist:

An Agentic Multimodal AI Assistant that Turns Lectures into Adaptive Study Material

By

Iftikhar Ahmed — ID # 2121019642
Salman Shafi — ID # 2211386042
Jarinah Tasnim — ID # 2121026642

Supervised by

Dr. Adnan Areefen
Department of Electrical and Computer Engineering
North South University

Submitted in partial fulfillment of the requirements
of the degree of Bachelor of Science in Computer Science and Engineering

September 6, 2026

Department of Electrical and Computer Engineering
North South University

APPROVAL

Iftikhar Ahmed ID # 2121019642
Salman Shafi ID # 2211386042
Jarinah Tasnim ID # 2121026642

from the Department of Electrical and Computer Engineering, North South University
have worked on the Senior Design Project titled:

"LectureAssist: An Agentic, Multimodal AI Assistant that Turns Lectures into Adaptive Study Material"

under the supervision of **Dr. Adnan Areefen** in partial fulfillment of the requirement for the degree of Bachelor of Science in Computer Science and Engineering and has been accepted as satisfactory.

Supervisor's Signature

.....

Dr. Adnan Areefen
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh.

Chairman's Signature

.....

Dr. Mohammad Abdul Matin
Professor
Department of Electrical and Computer Engineering
North South University
Dhaka, Bangladesh.

DECLARATION

It is to declare that this project is our original work. No part of this work has been submitted elsewhere, partially or entirely, for the award of any other degree or diploma. All project related information will remain confidential and shall not be disclosed without the formal consent of the project supervisor.

Relevant previous works presented in this report have been adequately acknowledged and cited. The plagiarism policy, as stated by the supervisor, has been maintained.

Students' Names & Signatures

1. **Iftikhar Ahmed**

- -

2. **Salman Shafi**

- -

3. **Jarinah Tasnim**

- -

Acknowledgements

This work would not have been possible without the input and support of many people over the last two trimesters. We would like to express our gratitude to everyone who contributed to it in some way or other.

First and foremost, we would like to thank our supervisor, **Dr. Adnan Areefen**, Department of Electrical and Computer Engineering, North South University, for his guidance throughout this project. His insistence that the evaluation be made rigorous that the generation prompts be shown rather than described, that chain-of-thought and program-of-thought baselines be compared directly against our agentic pipeline, and that question generation and answer generation be treated as two separate agents whose answers must be cross-checked fundamentally shaped this work, and turned it from a demonstration into a study.

Our sincere gratitude goes to the Department of Electrical and Computer Engineering, North South University, for providing the academic environment and the resources that made this project possible.

We are also thankful to the open-source community, whose freely available models and tools — Gemma 3, Ollama, Whisper, EasyOCR, Sentence-Transformers and FAISS — are the foundation on which this system is built, and without which a project of this scope could not have been undertaken without significant funding.

Last but not the least, we owe a great deal to our families, including our parents, for their unconditional love and immense emotional support.

Abstract

Recorded lectures and digital course material are now abundant, but the tools that help students convert that material into practice and self assessment are not. Existing study aids either require the student to paste in clean text by hand, or depend on paid cloud APIs that are impractical in bandwidth limited and cost-sensitive settings. This report presents **LectureAssist**, an end-to-end agentic AI system that turns a raw lecture video or an academic document into structured study and assessment material without any proprietary cloud service. The system couples a dual-channel extraction layer scene-adaptive optical character recognition that distinguishes slides, blackboards, whiteboards and on-screen text, combined with Whisper-based speech transcription to a multi-agent generation pipeline that plans, retrieves, generates, validates and refines five types of quiz questions, all served by locally hosted Gemma 3 models through the Ollama runtime on a single consumer-grade GPU. A retrieval augmented chat interface and an adaptive difficulty engine that tracks per topic performance complete the loop from raw media to personalised assessment. The system was evaluated on a large scale study in which multiple-choice questions were generated from a college-level calculus chapter across three difficulty levels, and scored by an independent, larger judge model on question quality, grounding, diversity and separately — *answer correctness*, using a second Answer Generator agent that re-solves each question without seeing the marked option. The work demonstrates that high quality, personalised educational AI is achievable on accessible hardware, and that question generators must be judged on answer correctness and diversity, not on fluency alone.

Table of Contents

List of Figures

List of Tables

Chapter 1

Introduction

This chapter motivates the project, states its purpose and contributions, and outlines the structure of the remainder of the report.

1.1 Background and Motivation

The proliferation of recorded lectures and digital course material has created a fundamental imbalance in self directed learning: students have more content than ever, but fewer tools with which to extract, consolidate and assess their understanding of it. A single semester course now routinely produces dozens of hours of recorded lectures, slide decks and reference chapters. Reviewing that material passively re-watching a video, re-reading a chapter is one of the least effective study strategies available. Decades of work in the learning sciences instead point to *retrieval practice*: repeatedly testing oneself on the material produces substantially more durable learning than re-exposure to it. The obstacle is supply. Producing good practice questions is slow, expert work, and the questions that do exist are concentrated at the end of textbook chapters, are not tied to what a particular instructor actually emphasised in class, and run out long before a student has mastered a weak topic.

Existing AI assisted study tools address this gap only partially. Text based chatbots require the student to manually transcribe or paste lecture content before they can help, which puts the burden of the hardest step getting the lecture out of the video and into text back on the student. Cloud dependent question generation services remove that burden, but they are inaccessible in bandwidth limited regions, they charge per token at a scale that makes routine practice expensive, and they require sending course material which may be copyrighted or institutionally restricted to a third party server. Crucially, none of these tools close the full loop from raw instructional media, through intelligent content understanding, to graded and personalised assessment without human intervention at each stage.

There is also a quality problem that is easy to overlook. A large language model asked to write a multiple choice question will almost always produce something that *reads* like

a good question. Whether the option it marks as correct actually *is* correct is a separate matter, and one that fluency based evaluation cannot detect. A study tool that confidently teaches a student the wrong answer is worse than no tool at all. Any serious system for automatic question generation therefore has to be evaluated on answer correctness, and on whether it keeps producing genuinely different questions rather than paraphrasing the same one, not merely on whether its output is well written.

1.2 Problem Statement

The problem this project addresses can be stated precisely. Given a lecture delivered as a video, in which the instructional content is distributed across speech, slides and board writing, and given no budget for commercial inference, produce assessment material that is grounded in that specific lecture, controlled for difficulty, verified for answer correctness, and varied enough that a student can practise on it repeatedly without meeting the same question twice.

Each clause in that statement rules out an existing class of solution.

Grounded in that specific lecture. A general-purpose model asked to write calculus questions will write calculus questions, but it will write them about calculus in general rather than about the material a particular instructor covered and emphasised. This is the difference between a question bank and a study aid. Meeting this clause requires the lecture content to be extracted, indexed and retrievable, which is why roughly half of the system described in Chapter 3 is concerned with getting content out of media rather than with generating anything.

Across speech, slides and board writing. Transcription alone recovers what was said and silently discards what was written. In technical subjects that is precisely the wrong half to lose, because the derivations, notation and worked examples are on the board rather than in the sentence. A system that reads only audio will produce questions about the discussion around a derivation while remaining unaware of the derivation itself.

With no budget for commercial inference. This constraint is economic in origin and technical in consequence. It fixes the generator at a model small enough to serve locally on free-tier hardware, and a small model produces malformed output, drifts off the requested difficulty, and repeats itself more than a frontier model does. Every one of those failure modes has to be engineered against rather than avoided by paying for a larger model.

Verified for answer correctness. A question whose marked option is wrong is not merely useless but harmful, and no amount of fluency reveals the error. Meeting this clause requires an independent check that does not share the generator's reasoning, which

is why answer generation is treated as a separate agent whose verdict is compared against the generator's.

Varied enough to practise on repeatedly. A generator that produces ten excellent but near-identical questions has failed at the task even though it would score well on any per-question quality metric. This clause is what forces diversity to be measured at the level of the corpus rather than the item, and it turns out to be the clause on which the pipelines in this study actually separate.

1.3 Objectives

The project set out to meet six objectives, each of which maps onto a component of the delivered system and onto a section of this report.

1. To build a dual-channel extraction layer that recovers instructional content from both the visual and the audio channel of a lecture recording, and to fuse the two into a single timestamped representation (Sections 3.1.2 and 3.1.1).

2. To make that representation searchable, so that any downstream component can retrieve the passage supporting a claim and any generated item can be traced back to its source moment (Section 3.1.3).

3. To design a multi-agent generation pipeline that decomposes question writing into planning, retrieval, drafting, validation and remediation, with quality control applied per question rather than per batch (Section 3.3.1).

4. To treat answer correctness as a property requiring independent verification rather than an assumption, by cross-checking every generated answer against a separate solving agent (Section 3.3.5).

5. To close the loop from assessment back to practice, by grading free-text responses and steering subsequent quizzes toward the topics on which a student performs worst (Section 3.3.7).

6. To evaluate the resulting system against a controlled baseline on measures capable of discriminating between them, and to report the outcome whether or not it favours the more complex design (Chapter 4).

1.4 Scope and Delimitations

Stating what the project does not attempt is as useful as stating what it does, and the boundaries below were fixed early and held throughout.

The system targets a **single user on a single machine**. It is not a multi-tenant service, it has no authentication layer, and no work was done on concurrency, rate limiting or access control. These are real requirements for a deployed product and they are deliberately outside the scope of a system built to demonstrate and evaluate a generation pipeline.

The generator is **fixed to one open-weight model family**. No model was trained or fine-tuned. The question of whether a fine-tuned generator would outperform a prompted one is interesting and is not asked here, because fine-tuning requires labelled data and compute that the zero-cost constraint excludes.

Evaluation is confined to **multiple-choice questions on one document**. The system generates five question types, but only multiple choice supports the independent re-solving check that the correctness measurement depends on, and only one source document was evaluated at the scale required for the comparison to be meaningful. Chapter 8 treats the resulting limits on generality explicitly rather than leaving them implied.

Finally, all quality judgements in this report are made by **models rather than by people**. A human study would anchor those judgements to ground truth and was beyond the time available. Where this bounds a conclusion, the bound is stated at the point the conclusion is drawn.

1.5 Purpose and Goal of the Project

The goal of this project is to build and rigorously evaluate **LectureAssist**, a fully local, API free system that ingests a lecture video or an academic document and autonomously produces validated, difficulty controlled assessment material, together with a retrieval grounded chat interface over the same content.

The system is built around three ideas working in concert. First, a *dualchannel extraction* layer simultaneously processes the visual channel of a lecture using scene type adaptive OCR that distinguishes slides, blackboards, whiteboards and on-screen text, each of which needs different image preprocessing and its audio channel via Faster Whisper transcription, producing a unified, timestamped lecture timeline. Second, a *multi-agent generation pipeline* decomposes quiz creation into specialist agents for planning, retrieval, generation, validation and targeted refinement, enabling structured quality control at the level of the individual question. Third, an *adaptive difficulty engine* maintains per topic performance histories across sessions and steers question allocation toward a student's weak areas. The entire system runs on locally hosted Gemma 3 models through the Ollama runtime, requires no external API keys, and is packaged for single GPU deployment on Google Colab with Google Drive session persistence.

The key contributions of this work are:

- A unified multi modal lecture understanding pipeline that fuses scene adaptive OCR with filtered speech transcription into a synchronised content timeline, supporting

both video and document inputs.

- A multi-agent quiz architecture implementing a Plan → Retrieve → Generate → Validate → Refine cycle across five question types (MCQ, True/False, Fill-in-the-Blank, Short Answer and Flashcard), with per-question difficulty, grounding and instruction-compliance validation, and a validator that chooses *how* to remediate a rejected question rather than merely rejecting it.
- An adaptive assessment engine that tracks per-topic student performance across sessions and personalises question difficulty and distribution without manual configuration.
- A multi-tier grading system combining deterministic matching, fuzzy string similarity and LLM-based rubric evaluation for free-text short answers, with targeted concept-level feedback on incorrect responses.
- A fully local, API-free deployment on consumer hardware using the Ollama inference runtime, with RAG-enabled semantic chat and persistent session storage.
- A large-scale evaluation harness that compares the agentic pipeline against three direct-prompting baselines, plain, chain-of-thought and program-of-thought, and which, unlike prior automatic question generation evaluations, separates *question quality* from *answer correctness* by having a second, larger Answer Generator agent independently re-solve every generated question without seeing the marked option.

1.6 Organization of the Report

The remainder of this report is organised as follows. Chapter 2 provides the background needed to follow the rest of the report, large language models and the prompting strategies used as baselines, retrieval-augmented generation, agentic architectures, speech recognition and OCR, then reviews the research literature and the existing commercial applications closest to this work, and identifies the gaps this project addresses. Chapter 3 describes the system design: the extraction layer, the agent architecture, the four question-generation pipelines with their prompts given verbatim, the two-agent answer cross-check, and the software components used to build it. Chapter 4 presents the evaluation: the environment, the metrics and exactly how each is computed, the large-scale multiple-choice results, and a discussion of what they show, including a failure mode of the agentic pipeline that the evaluation exposed. Chapter 5 discusses the societal, ethical and environmental implications and the design constraints they impose. Chapter 6 presents the project plan and budget. Chapter 7 maps the work onto the complex engineering problem and activity attributes. Chapter 8 summarises the project, states its limitations honestly, and sets out the work that follows from it.

Chapter 2

Background and Literature Review

This chapter presents the theoretical foundation and reviews the existing literature related to large language model based educational question generation. Recent advances in large language models have significantly improved the ability of intelligent systems to generate educational content, particularly multiple choice questions that support teaching, learning, and assessment. At the same time, ensuring that generated questions are pedagogically meaningful, cognitively appropriate in the sense of Bloom's taxonomy and its later revision [1, 2], and aligned with educational objectives remains a challenging research problem, as the surveys of the field make clear [3]. Consequently, recent studies have shifted their focus from simply generating fluent questions toward developing structured generation frameworks and comprehensive evaluation methodologies.

This chapter first introduces the fundamental concepts required to understand the proposed system, including large language models, prompting techniques, agent based architectures, and evaluation strategies for educational question generation. It then reviews recent research that investigates controlled question generation using large language models, with particular emphasis on hybrid multi agent generation frameworks and educational question evaluation benchmarks. Existing educational applications are subsequently discussed before identifying the research gaps that motivate the proposed work presented in this thesis.

2.1 Preliminaries

Educational question generation using large language models integrates concepts from natural language processing, prompt engineering, intelligent agent systems, and educational assessment. Understanding these foundational concepts is essential for designing systems that generate high quality assessment items while maintaining pedagogical relevance and consistency. This section introduces the key technologies and methodologies that underpin the proposed framework, including large language models, prompting strategies, agent

based architectures, and evaluation techniques. These concepts provide the theoretical foundation for the methodology presented in later chapters.

2.1.1 Large Language Models and Prompting

Large Language Models (LLMs) have emerged as one of the most significant advancements in artificial intelligence, demonstrating remarkable capabilities in understanding, reasoning over, and generating natural language. Built upon the Transformer architecture [4], LLMs are pretrained on extensive text corpora using self supervised learning [5], enabling them to capture complex linguistic patterns, semantic relationships, and contextual information. Through subsequent instruction tuning and alignment techniques, these models can perform a wide variety of language tasks including summarization, translation, question answering, content generation, and educational assistance without requiring task specific model architectures. Their versatility has led to widespread adoption across educational, industrial, and research applications, particularly in tasks that require generating coherent and contextually relevant textual content. [6, 7]

The emergence of LLMs has fundamentally changed the landscape of automatic educational question generation. Earlier approaches often relied on handcrafted linguistic rules [8] or supervised neural models trained for a single task [9, 10], limiting their adaptability across different educational domains. In contrast, modern LLMs can generate diverse assessment items from instructional materials using natural language instructions alone. This capability allows a single model to generate questions of different formats, difficulty levels, and cognitive requirements without extensive retraining. Consequently, recent research has increasingly focused on designing effective interaction strategies that enable language models to generate pedagogically meaningful and educationally appropriate assessment items rather than developing specialized generation models for each individual task. [6, 7]

The quality of responses produced by an LLM depends not only on the model itself but also on how the task is presented. Prompt engineering refers to the process of designing structured instructions that guide the model toward producing outputs aligned with specific objectives. In educational question generation, prompts commonly specify the source material, target knowledge concepts, question type, difficulty level, and expected output format. Well designed prompts reduce ambiguity, improve consistency, and help ensure that generated questions remain relevant to the learning objectives. Recent systems further enhance this process by dynamically constructing prompts using contextual information generated during intermediate processing stages, allowing the model to produce more accurate and context aware assessment items. [6, 7]

Several prompting techniques have been proposed to further improve educational question generation. One widely adopted approach is Chain of Thought prompting [11, 12], which encourages the model to explicitly perform intermediate reasoning before generating the final output. Rather than producing a question immediately, the model first analyzes

the instructional content, identifies important concepts, determines appropriate learning objectives, and then constructs a question that reflects the intended educational outcome. This reasoning process improves the generation of higher order questions requiring conceptual understanding or logical inference while reducing inconsistencies in the generated assessment items. Modern educational generation frameworks frequently employ Chain of Thought prompting during planning and evaluation stages to improve both reliability and pedagogical quality. [7]

Instruction tuning makes models markedly more responsive to such directives [13]. Another important prompting strategy is persona based prompting, where the language model is instructed to assume the role of an experienced teacher, curriculum designer, or assessment specialist. Assigning a professional role encourages the model to generate questions that more closely resemble those created by human educators while adhering to established educational practices. Recent studies also incorporate self critique prompting, which introduces an internal evaluation stage after question generation. During this stage, the language model reviews its own output by examining question clarity, answer correctness, distractor quality, and alignment with instructional objectives before revising the question if necessary. This self-critique-and-revise pattern is the educational instance of a general technique in which a model iteratively refines its own output against feedback [14, 15], and it is the pattern the validate-and-retry loop of this project adopts. Together, these prompting strategies improve the controllability, consistency, and educational quality of modern large language model based question generation systems. [7]

2.1.2 Agent Based Architectures

Early large language model based educational question generation systems primarily relied on a single prompt to transform instructional content into assessment questions. Although these systems demonstrated impressive language generation capabilities, they often struggled to maintain consistency, accurately control cognitive complexity, and verify the correctness of generated questions. A single language model was expected to understand the source material, identify important concepts, generate meaningful questions, construct plausible distractors, and evaluate the quality of the final output within a single interaction. As the complexity of educational tasks increased, this monolithic approach frequently produced questions with ambiguous wording, incorrect answers, or distractors that failed to effectively distinguish different levels of student understanding. Consequently, recent research has shifted towards agent based architectures that decompose the overall generation process into multiple coordinated subtasks.

An agent based architecture consists of several specialized agents that collaborate to accomplish a complex task [16, 17]. Rather than assigning every responsibility to a single language model interaction, each agent performs a dedicated function while exchanging information with other agents throughout the workflow. Individual agents may be responsible for analyzing instructional content, identifying learning objectives, generating

questions, evaluating assessment quality, or formatting the final output. This modular design improves transparency, allows intermediate outputs to be verified before proceeding to subsequent stages, and enables individual components to be refined independently without affecting the entire system. Such characteristics make agent based architectures particularly suitable for educational applications where accuracy, consistency, and pedagogical quality are essential.

A representative example of this approach is the ReQUESTA framework, which employs a hybrid multi agent architecture for automatic multiple choice question generation. Instead of relying solely on a language model, the framework combines deterministic rule based components with LLM powered reasoning agents. Rule based modules perform structured operations such as document segmentation, workflow coordination, and output formatting, while specialized language model agents perform higher level reasoning tasks including concept identification, question generation, answer verification, and quality evaluation. This hybrid design leverages the reliability of traditional algorithms together with the flexibility and reasoning capability of modern large language models, resulting in more robust educational question generation.

Within the ReQUESTA workflow, the instructional material is first divided into manageable text segments before being analyzed by a planning agent. The planning agent identifies important concepts, determines suitable learning objectives, and prepares a structured generation plan for subsequent stages. Based on this plan, specialized generation agents create different categories of questions that target various cognitive skills. A dedicated evaluation agent then reviews each generated question by examining its clarity, correctness, and educational relevance. If deficiencies are identified, the question is returned to the corresponding generation agent for revision, creating an iterative refinement process that continues until the generated assessment satisfies predefined quality requirements. Finally, formatting components standardize the presentation of the questions for learners.

The adoption of agent based architectures demonstrates that improvements in educational question generation depend not only on increasingly capable language models but also on carefully designed workflows that coordinate reasoning, generation, and evaluation. By separating complex educational tasks into specialized components, these architectures provide greater control over question quality, improve alignment with learning objectives, and facilitate the development of scalable educational assessment systems. Consequently, agent based workflows have become an important direction in recent research on large language model based educational question generation.

2.1.3 Evaluation of Educational Question Generation

Evaluating automatically generated educational questions is considerably more challenging than evaluating general text generation tasks. While traditional natural language generation metrics such as BLEU [18], ROUGE [19], and METEOR measure lexical similarity

between generated text and reference answers, they often fail to capture the educational quality of assessment items. Multiple valid questions can be generated from the same instructional material using different wording, reasoning processes, or assessment formats while remaining equally effective for learning. Consequently, relying solely on text similarity metrics may incorrectly penalize high quality questions that differ from reference examples, making conventional evaluation methods insufficient for educational question generation.

Recent research has therefore shifted towards pedagogically oriented evaluation frameworks that assess whether generated questions satisfy educational objectives rather than merely matching reference text. Important evaluation criteria include the correctness of the generated answer, alignment between the question and the intended knowledge point, appropriateness of the question type, clarity of language, cognitive complexity, and the quality of distractors in multiple choice questions. These dimensions provide a more comprehensive assessment of educational usefulness by considering both linguistic quality and instructional effectiveness.

One notable contribution in this area is EQGBench, a benchmark specifically developed for evaluating educational question generation using large language models. Unlike conventional benchmarks, EQGBench introduces a multidimensional evaluation framework that measures educational quality from several complementary perspectives. The benchmark evaluates knowledge point alignment, question type alignment, overall question quality, solution quality, and competence oriented guidance. Together, these dimensions assess whether generated questions accurately reflect instructional content, employ appropriate assessment formats, provide correct and complete solutions, and promote meaningful learning outcomes rather than simple factual recall. This multidimensional framework offers a more reliable assessment of educational question generation than traditional text similarity metrics alone. [6]

Another important advancement is the use of large language models as evaluators. Instead of depending exclusively on automatic similarity metrics or manual expert review, recent systems employ language models to assess generated questions according to predefined educational criteria. Acting as impartial evaluators [20], these models examine question clarity, factual correctness, relevance to the source material, distractor plausibility, and overall pedagogical quality. This approach enables scalable evaluation while maintaining consistency across large numbers of generated assessment items. Human expert evaluation remains the gold standard for educational assessment; however, LLM based evaluation has emerged as a practical alternative for benchmarking and iterative system development.

The increasing emphasis on comprehensive evaluation reflects the growing maturity of educational question generation research. As generation quality continues to improve through advances in large language models and agent based workflows, evaluation methodologies have likewise evolved to measure educational effectiveness rather than linguistic similarity alone. These developments provide a stronger foundation for designing intel-

ligent assessment systems capable of generating reliable, pedagogically meaningful, and cognitively appropriate questions for diverse educational settings. [6, 7]

2.2 Literature Review

Automatic educational question generation has been an active area of research for several decades, evolving alongside advances in natural language processing and artificial intelligence. Early approaches relied primarily on manually designed linguistic rules and handcrafted templates to transform declarative sentences into questions. Although these methods were computationally efficient and generated grammatically correct questions within restricted domains, they required extensive human effort and exhibited limited adaptability to diverse educational materials. As machine learning techniques matured, researchers gradually shifted towards data driven approaches capable of learning question generation patterns directly from large annotated datasets. More recently, the emergence of large language models has fundamentally transformed the field by enabling the generation of high quality, context aware, and pedagogically meaningful assessment items through natural language instructions rather than task specific model architectures.

Despite these advances, producing educational questions that accurately assess learning outcomes remains a challenging problem. High quality assessment items must not only be grammatically correct but also align with instructional objectives, target appropriate cognitive levels, contain unambiguous answers, and provide effective distractors for multiple choice questions. Consequently, recent research has increasingly focused on developing structured generation frameworks and comprehensive evaluation methodologies that improve both the quality and educational value of automatically generated questions.

This section reviews the major developments in educational question generation, beginning with traditional approaches before discussing recent large language model based methods. Particular emphasis is placed on two representative studies that address complementary aspects of the problem: the ReQUESTA framework, which proposes a hybrid multi agent architecture for controlled multiple choice question generation, and EQG-Bench, which introduces a comprehensive benchmark for evaluating the educational quality of questions generated by large language models. Together, these studies represent important advancements in both the generation and evaluation of intelligent educational assessment systems.

2.2.1 Evolution of Educational Question Generation

The development of automatic educational question generation has closely followed the evolution of natural language processing techniques. Early research primarily relied on rule based systems that transformed declarative sentences into interrogative forms using handcrafted linguistic rules, syntactic parsing, and manually designed templates. These approaches generated grammatically correct questions within restricted domains and of-

fered high interpretability because the generation process was explicitly defined by human experts. However, developing and maintaining large collections of language rules required substantial manual effort, while the resulting systems often struggled to generalize across different subjects, writing styles, and educational contexts. As a result, rule based methods exhibited limited scalability and were unsuitable for generating diverse assessment items from complex instructional materials.

The availability of large educational datasets and advances in machine learning led researchers to adopt data driven approaches for question generation. Sequence-to-sequence neural networks enabled models to learn mappings between instructional content and corresponding questions directly from annotated examples rather than relying on manually designed rules. Attention mechanisms further improved these models by allowing them to focus on the most relevant portions of the input text during generation. Compared with traditional rule based systems, neural approaches produced more fluent and diverse questions while requiring significantly less manual feature engineering. Nevertheless, their performance depended heavily on the availability of large, high quality training datasets and they often struggled to generate questions requiring complex reasoning or deep conceptual understanding.

The introduction of Transformer based architectures marked another major advancement in automatic question generation. Transformer models demonstrated superior contextual understanding through self attention mechanisms, enabling them to capture long range dependencies and semantic relationships more effectively than recurrent neural networks. Pretrained language models such as BERT, T5, and BART further improved generation quality by leveraging knowledge acquired from large scale pretraining before being adapted to educational tasks. These models significantly enhanced grammatical fluency, contextual relevance, and the diversity of generated questions. However, most Transformer based systems still required task specific fine tuning and were generally designed for fixed question generation objectives, limiting their flexibility across different educational settings.

Recent progress in large language models has fundamentally transformed educational question generation. Unlike earlier approaches that required dedicated training for individual tasks, modern LLMs can generate assessment items through carefully designed natural language prompts without extensive task specific optimization. This capability enables a single model to produce multiple question formats, adjust difficulty levels, and incorporate reasoning processes using prompt engineering techniques alone. Contemporary research has therefore shifted its emphasis from improving language generation itself toward developing structured workflows, agent based architectures, and comprehensive evaluation methodologies that ensure generated questions satisfy pedagogical objectives. These developments represent a transition from purely language focused generation toward intelligent educational assessment systems capable of producing reliable, context aware, and pedagogically meaningful questions across diverse learning environments. [6, 7]

2.2.2 Large Language Model Based Educational Question Generation

The emergence of large language models has significantly advanced the field of educational question generation by enabling the production of coherent, context aware, and pedagogically relevant assessment items without extensive task specific training. Unlike earlier neural approaches that required fine tuning for individual datasets or question types, modern LLMs leverage knowledge acquired during large scale pretraining to perform educational tasks through carefully designed natural language prompts. This capability has greatly improved the adaptability of question generation systems, allowing them to operate across multiple academic disciplines and educational settings while requiring only minimal modifications to the input instructions.

Recent research has increasingly focused on improving the controllability of question generation rather than solely enhancing linguistic quality. One representative study is the ReQUESTA framework, which proposes a hybrid multi agent architecture for generating cognitively diverse multiple choice questions. The framework recognizes that educational assessment involves several interconnected tasks, including identifying learning objectives, selecting appropriate concepts, constructing meaningful questions, generating plausible distractors, and verifying the quality of the final assessment item. Instead of assigning all these responsibilities to a single language model interaction, ReQUESTA distributes them among multiple specialized agents that collaborate throughout the generation process. This modular design enables greater control over each stage of question generation while reducing errors that commonly occur in single prompt systems. [7]

The ReQUESTA framework combines deterministic rule based processing with the reasoning capability of large language models. Rule based components manage operations such as document segmentation, workflow coordination, and output formatting, while specialized LLM agents perform higher level reasoning tasks including concept identification, question generation, quality evaluation, and revision. The workflow begins by dividing instructional material into manageable text segments, after which a planning agent analyzes each segment to identify key concepts and determine suitable learning objectives. Based on this analysis, dedicated generation agents create multiple choice questions targeting different cognitive skills before an evaluation agent reviews each question for correctness, clarity, and educational relevance. Questions that do not satisfy predefined quality criteria are automatically revised through an iterative feedback process until acceptable quality is achieved. This structured workflow improves both the reliability and consistency of generated assessment items compared with conventional single prompt approaches. [7]

Another important contribution of ReQUESTA is its use of advanced prompting strategies throughout the generation pipeline. The framework incorporates Chain of Thought reasoning to encourage systematic analysis before question generation, persona based prompting to emulate experienced educators during assessment design, and self critique mechanisms that enable the language model to evaluate and refine its own outputs. By integrating these techniques within a coordinated multi agent workflow, the framework

produces questions with clearer wording, more accurate answers, and higher quality distractors. Experimental results reported in the study demonstrate that this hybrid architecture consistently outperforms conventional zero shot prompting approaches in both automated evaluation and expert assessment, highlighting the importance of structured workflows in educational question generation. [7]

The findings of the ReQUESTA study demonstrate that advances in educational question generation depend not only on increasingly capable language models but also on carefully designed system architectures that coordinate planning, reasoning, generation, and evaluation. These observations have influenced recent research by encouraging the development of modular, collaborative frameworks that improve both the pedagogical quality and reliability of automatically generated educational assessments.

2.2.3 Evaluation Frameworks for Educational Question Generation

While recent advances in large language models have significantly improved the generation of educational questions, evaluating the quality of these questions remains an equally important challenge. Unlike conventional natural language generation tasks, educational question generation cannot be assessed solely through lexical similarity between generated and reference texts. Multiple questions may correctly assess the same learning objective despite differing in wording, structure, or reasoning process. Consequently, traditional evaluation metrics such as BLEU, ROUGE, and METEOR often fail to reflect the true pedagogical value of generated assessment items. This limitation has motivated researchers to develop evaluation frameworks that measure educational effectiveness rather than textual similarity alone.

A notable contribution in this direction is EQGBench, a benchmark specifically designed for evaluating educational question generation using large language models. The benchmark was developed to address the absence of standardized evaluation methodologies capable of measuring both linguistic quality and educational relevance. Rather than relying exclusively on automatic similarity metrics, EQGBench introduces a multidimensional evaluation framework that examines several complementary aspects of educational assessment. These dimensions include knowledge point alignment, question type alignment, question quality, solution quality, and competence oriented guidance. Together, these criteria provide a comprehensive assessment of whether generated questions accurately reflect instructional content while supporting meaningful learning outcomes. [6]

Knowledge point alignment evaluates whether a generated question correctly targets the intended concepts presented in the instructional material. Question type alignment determines whether the generated assessment follows the desired format, such as multiple choice, short answer, or explanatory questions. Question quality focuses on linguistic clarity, logical consistency, and the absence of ambiguity, while solution quality measures the correctness and completeness of the accompanying answers or explanations. Competence oriented guidance extends the evaluation beyond factual correctness by assessing whether

the generated questions encourage reasoning, conceptual understanding, and higher order thinking skills. Collectively, these dimensions provide a richer understanding of educational quality than conventional text based evaluation metrics. [6]

To validate the proposed benchmark, the authors constructed a comprehensive evaluation dataset consisting of educational materials from mathematics, physics, and chemistry. The benchmark was then used to evaluate numerous state of the art large language models under a standardized experimental protocol. The results demonstrated substantial variation in educational performance among different models despite many of them achieving comparable scores on traditional language generation benchmarks. These findings highlight that strong general language capabilities do not necessarily translate into high quality educational assessment generation, emphasizing the importance of specialized evaluation frameworks tailored to educational applications. [6]

Another significant contribution of recent evaluation research is the growing adoption of large language models as automated evaluators. Instead of depending exclusively on human experts or lexical similarity metrics, language models can assess generated questions according to predefined educational criteria such as clarity, correctness, relevance, and pedagogical appropriateness. Although human evaluation remains the most reliable method for assessing educational quality, LLM based evaluation offers a scalable and consistent alternative for benchmarking large numbers of generated questions during system development. The integration of comprehensive evaluation frameworks such as EQGBench with automated LLM based assessment represents an important step toward developing reliable, standardized methodologies for educational question generation research. [6]

2.3 Research Gap

The literature demonstrates that recent advances in large language models have significantly improved the generation and evaluation of educational questions. The ReQUESTA framework shows that hybrid multi agent architectures can produce more reliable and cognitively diverse multiple choice questions by separating planning, generation, evaluation, and refinement into specialized components. Similarly, EQGBench establishes that evaluating educational question generation requires multidimensional assessment criteria that consider pedagogical quality in addition to traditional language generation metrics. Together, these studies represent important progress toward intelligent educational assessment systems.

Despite these contributions, several research challenges remain. Existing generation frameworks primarily focus on producing high quality assessment items from textual content but provide limited support for integrating diverse educational resources into a unified generation pipeline. Many systems assume that the instructional content has already been prepared in a structured textual format, making them less suitable for practical educational environments where learning materials may originate from lecture notes, presentation slides, or other unstructured documents.

Furthermore, although recent studies emphasize question quality and evaluation, relatively little attention has been given to developing end to end educational assistance systems that combine document understanding, intelligent question generation, automated answer production, and comprehensive evaluation within a single workflow. In many existing approaches, these components are investigated independently rather than being integrated into a cohesive educational framework capable of supporting both instructors and learners.

Another limitation is that current research predominantly evaluates generated questions using benchmark datasets under controlled experimental settings. While these evaluations provide valuable insights into model performance, they may not fully reflect real world educational scenarios involving diverse instructional materials, varying document structures, and different learning objectives. Consequently, there remains a need for practical systems that effectively combine large language models with structured generation workflows while maintaining high educational quality across different types of learning resources.

Motivated by these research gaps, this thesis proposes a large language model based educational question generation framework that integrates document processing, intelligent question generation, answer generation, and automated evaluation into a unified system. The proposed approach aims to generate contextually relevant and pedagogically meaningful assessment items while providing a flexible and scalable workflow that can be applied to a wide range of educational materials.

2.3.1 Comparison of Existing Systems

Table 2.1 places the systems and frameworks discussed above beside one another on the dimensions that matter for this project. The comparison is restricted to properties that can be established from the published descriptions of each system, and the final row describes the system built in this work.

Table 2.1: Comparison of related systems on the dimensions relevant to this project. Entries marked n/a are not applicable to that class of system.

System	Input	Multi-agent	Local	Answer check	Evaluation
Rule based generators	Clean text	No	Yes	No	Lexical overlap
Neural sequence to sequence	Clean text	No	Yes	No	BLEU, ROUGE
Prompted large language model	Clean text	No	No	No	Judged quality
Multi-agent authoring frameworks	Clean text	Yes	No	Partial	Expert rubric
Benchmark driven evaluation	Generated items	n/a	No	No	Multi-dimensional
LectureAssist	**Video, audio, board, documents**	**Yes**	**Yes**	**Yes, separate agent**	**Corpus level plus judged**

Three observations follow from that table, and together they define the space this project occupies.

First, every prior row assumes **clean text as input**. The step of recovering instructional content from a recorded lecture is treated as somebody else's problem, which means the burden of transcription falls on the student who is least equipped to carry it. A system that accepts a video and produces assessment material without manual preparation is doing work that the surveyed literature consistently assumes away.

Second, the systems that adopt **multi-agent decomposition** are also the systems that depend on commercial inference. This is not accidental: decomposing generation into planning, drafting, validation and refinement multiplies the number of model calls, and that multiplication is affordable when a frontier model is being billed per token by an institution but not when the target user is a student without a payment method. Whether the benefits of decomposition survive being run on a small local model is an open question, and it is the question Chapter 4 answers.

Third, **answer correctness is almost never verified independently**. Most evaluations score the question and take the marked answer on trust, which means a generator that writes fluent questions with wrong keys scores well. Separating question generation from answer generation, and requiring the two to agree before an item is trusted, is the clearest methodological gap in the surveyed work and is treated as a first-class requirement here.

2.3.2 Saturation of Judged Quality Metrics

One finding recurs across the evaluation literature and deserves separate treatment, because it shapes the entire design of this project's experiment.

When capable language models are scored by a judge on dimensions such as clarity, correctness, relevance or format compliance, the scores cluster near the top of whatever scale is used. Benchmarks that score generated questions on multi-point scales routinely report that their strongest models sit within a few hundredths of one another and within a few hundredths of the maximum, across most of the dimensions measured. The dimensions do not fail because they are badly designed; they fail because modern models genuinely are good at the surface properties those dimensions measure. Producing a clearly worded, correctly formatted, topically relevant question is no longer difficult.

The consequence for experimental design is that **a metric near its ceiling cannot discriminate**. If two systems both score above nine tenths of the available range, the gap between them is smaller than the noise in the judge, and no conclusion can be drawn from their ordering. An evaluation built only on such dimensions will report that every system is excellent and that none is distinguishable, which is a statement about the instrument rather than about the systems.

Escaping this requires measures that are not bounded above by model competence at surface fluency. Properties of the generated *corpus* rather than the individual item serve

this purpose well, because they measure something a fluent generator can still fail at: whether the questions are different from one another, whether they are actually supported by the source material, and whether the marked answers survive independent scrutiny. These properties do not saturate, they are computed rather than judged, and they are therefore not subject to the biases that affect a model acting as a rater. The evaluation designed in Chapter 4 reports judged dimensions for completeness but rests its conclusions on corpus-level measures for exactly this reason.

Chapter 3

Methodology

This chapter describes how LectureAssist is built: the overall system design and the flow of data through it, the software components chosen and why, and the implementation of each stage — extraction, retrieval, the multi-agent generation loop, the four question-generation pipelines whose prompts are given verbatim, the two-agent answer cross-check, and the grading and adaptation layers.

3.1 System Design

LectureAssist is organised as a pipeline of specialist agents coordinated by an orchestrator. The user supplies either a lecture video or a document (PDF, DOCX, PPTX); the system converts it into a single grounded content representation, and every downstream capability — summary, quiz, chat, adaptive practice — consumes that representation rather than the raw media. The design is deliberately staged so that each stage produces an artifact that can be inspected, cached and re-used, which is what makes the expensive stages (OCR, transcription) survivable on free-tier hardware.

3.1.1 Data flow

The end-to-end flow is as follows.

1. **Ingestion.** A video is sampled into frames and its audio track is demuxed; a document is parsed page by page.
2. **Dual-channel extraction.** The visual channel goes to the *Extractor* agent (scene-adaptive OCR); the audio channel goes to the *Transcriber* agent (Faster-Whisper). Documents bypass both and go to the *DocParser* agent.
3. **Fusion.** OCR blocks and transcript segments are merged in timestamp order into a single unified lecture timeline, so that what the lecturer *said* sits next to what they *wrote* at the same moment.

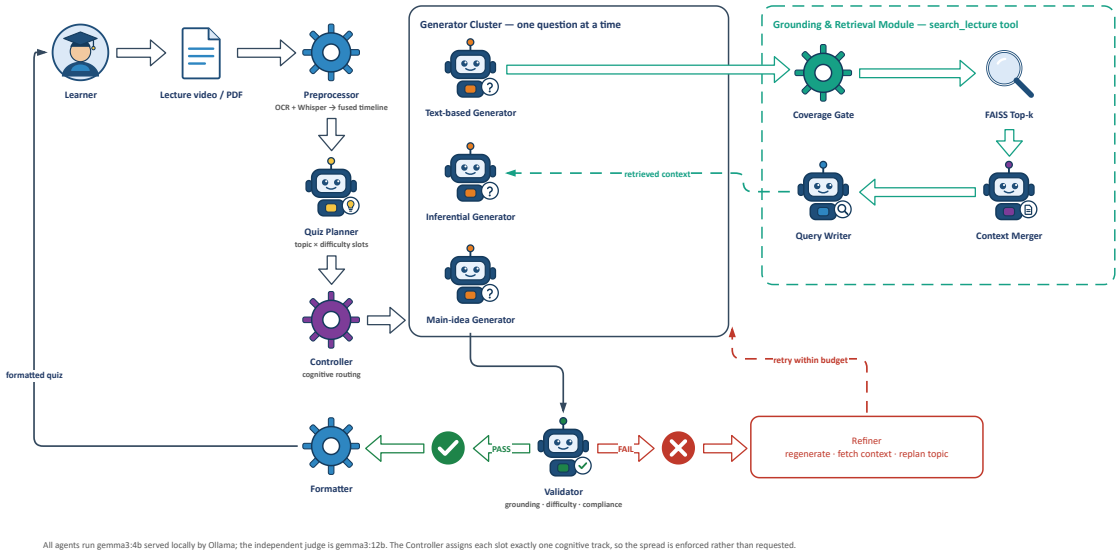

Figure 3.1: System architecture of LectureAssist. The orchestrator coordinates specialist agents across ingestion, representation and generation: the Controller routes each slot to one of three cognitive tracks, the Validator accepts or rejects each drafted question, and a rejected question is diagnosed and returned to the Refiner rather than discarded. The dashed module on the right is the model-directed `search_lecture` retrieval tool.

4. **Understanding.** The *Summary* agent produces a structured summary; the *RAG-Builder* agent chunks the content, embeds it and builds a vector index; the *Hints* agent flags passages the lecturer marked as exam-relevant.

5. **Generation.** The *Quiz Planner* allocates questions across topics and difficulties; a *Question Generator* drafts one question at a time; a *Validator* accepts or rejects it and chooses a remediation action; a *Refiner* applies it. This loop is the core of the system and is described in Section 3.3.1.

6. **Assessment and adaptation.** Answers are graded by a multi-tier grader; per-topic performance is written to a persistent profile, which the *Adaptive Quiz* agent uses to bias the next quiz toward weak topics.

3.1.2 Scene-adaptive extraction

A single OCR preprocessing recipe cannot handle a real lecture. White chalk on a dark board is a low-contrast, inverted-polarity image; a projected slide in a lit room is a bright, high-contrast one; a dark-theme slide deck looks superficially like a blackboard but is destroyed by the inversion that a blackboard requires. LectureAssist therefore *classifies each sampled frame first* and preprocesses it accordingly. Frames are routed by brightness and edge statistics into four classes — `slide`, `blackboard`, `whiteboard` and generic `scene` (on-screen text) — and each class gets its own contrast enhancement, thresholding and, where appropriate, polarity inversion before being passed to EasyOCR [21]. Consecutive frames whose extracted text is more than 78% similar are treated as the same board state

and deduplicated, so that a slide left on screen for two minutes contributes one block rather than sixty.

3.1.3 Retrieval

Every generated artifact is grounded by retrieval-augmented generation [22, 23]: the model is never asked to recall the lecture from its weights, only to work from passages retrieved out of the lecture itself.

The fused timeline is chunked and embedded with the Sentence-BERT model [24] `all-MiniLM-L6-v2` [25]. The vectors are L_2-normalised and indexed in FAISS [26] with an inner-product index, so that inner product equals cosine similarity (Eq. 3.1):

$$\mathrm{sim}(q, c) \;=\; \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\|\,\|\mathbf{e}_c\|} \;=\; \hat{\mathbf{e}}_q \cdot \hat{\mathbf{e}}_c, \qquad \hat{\mathbf{e}} = \frac{\mathbf{e}}{\|\mathbf{e}\|}, \tag{3.1}$$

where $\mathbf{e}_q, \mathbf{e}_c \in \mathbb{R}^{384}$ are the embeddings of the query and of a candidate chunk. Because every vector is L_2-normalised before insertion, the denominator is unity and a plain inner-product search returns exactly the cosine ranking, which is why no separate normalisation step is needed at query time. Retrieval returns the top $k = 5$ chunks together with their timestamps, which is what allows both the chat interface and a generated question to be traced back to the moment in the lecture they came from. Crucially, retrieval is exposed to the generator agent as a *tool* (`search_lecture`) rather than being applied unconditionally: the agent decides for itself whether it needs more evidence.

3.2 Hardware and/or Software Components

The system is written in Python and served as a Flask [27] API, with a single-page HTML/JavaScript front end that talks to it over an ngrok [28] tunnel; this split lets the heavy models run on a Colab [29] GPU while the interface runs in the student's own browser. Table 3.1 lists the components, the alternatives considered, and the reason for each choice. The overriding constraint throughout was that *nothing may require a paid API key*, since the target users are precisely those for whom a per-token bill is prohibitive.

Table 3.1: Software components, alternatives, and selection rationale.

Tool	**Function**	**Other similar tools**	**Why selected**
Gemma 3 (4B / 12B)	Generation, summarisation, grading, judging	GPT-4, Claude, Llama 3, Mistral	Open weights, runs locally, no API key; two sizes let the cheap model generate and the strong model judge.
Ollama	Local LLM serving	llama.cpp, vLLM, HF Transformers	One-line model pull, automatic GPU offload, OpenAI-style local HTTP API; vLLM needs more VRAM than a free T4 has.
Faster-Whisper (`small`)	Speech-to-text	OpenAI Whisper API, Vosk, wav2vec2	Same accuracy as reference Whisper at a fraction of the memory; runs offline. The API version is paid and cloud-bound.
EasyOCR (en, bn)	Board and slide text extraction	Tesseract, PaddleOCR, Google Vision	Deep-learning based, far better on handwriting than Tesseract; native Bengali support matters for local lectures; Google Vision is a paid cloud API.
Sentence-Transformers `all-MiniLM-L6-v2`	Embeddings for RAG and for evaluation metrics	OpenAI `text-embedding-3`, BGE, E5	384-dim, small and fast enough to co-exist with the LLM on one GPU; strong quality-per-byte; local.
FAISS	Vector index	Chroma, Pinecone, pgvector	In-process, no server, no network; exact inner-product search is more than sufficient at lecture scale.
PyMuPDF [30]	PDF/DOCX/PPTX parsing	pdfplumber, PyPDF2	Fastest reliable text-layer extraction; preserves page structure used for per-page source attribution.
Flask + ngrok	Backend API and public tunnel	FastAPI, Gradio, Streamlit	Minimal surface for a JSON API; ngrok exposes the Colab GPU to a browser UI running on the student's own machine.
SymPy	Building the verified answer key for calibration	Hand-computed answers, WolframAlpha	Answers are *machine-computed*, not hand-typed, so the calibration set cannot inherit a human arithmetic error.
Google Colab (T4) + Drive	Compute and persistence	Local GPU, RunPod, AWS	Free tier is the realistic deployment target; Drive persistence lets a run survive a runtime disconnect.

3.3 Hardware and/or Software Implementation

3.3.1 The agentic question-generation loop

The agentic Question Generator produces *one question at a time* through a multi-stage loop. The design decision that distinguishes it from a generate-then-filter pipeline is that the validator does not merely reject a bad question — it *diagnoses* it and chooses how the next attempt should differ. The stages are:

1. **Plan.** The Quiz Planner reads the lecture summary and distributes the requested questions across the topics that are actually present, each with a target difficulty, yielding a list of (topic, difficulty) slots.

2. **Retrieve.** Before drafting a slot, the generator inspects the content it holds and decides for itself whether it has enough material on that topic. If not, it issues its own retrieval query through the `search_lecture` tool to pull more context. This is genuine tool use: the decision to retrieve is the model's, not a fixed step in the pipeline.

3. **Generate.** It drafts exactly one question under a grounding rule — the question must be answerable using only the supplied content — rotating a question strategy on each attempt (concept-check, scenario, compare-and-justify) so that a retry is not merely a resample of the same prompt.

4. **Validate.** A separate call checks the draft on three axes — difficulty match, grounding, and instruction compliance — and returns `PASS` or `FAIL` together with a remediation `action`.

5. **Retry (loop back).** On `FAIL` the loop repeats within a retry budget, and the validator's chosen action decides the fix: `regenerate` (rewrite, with the failure reason fed back into the prompt), `fetch_context` (retrieve more evidence first, then retry), or `replan_topic` (abandon a topic that is not really in the lecture and swap to one that is).

6. **Override.** If the retry budget is exhausted, the best attempt so far is kept and explicitly *labelled* as an override. It is never silently recorded as a clean pass — otherwise the pipeline's own accept rate would be a fiction.

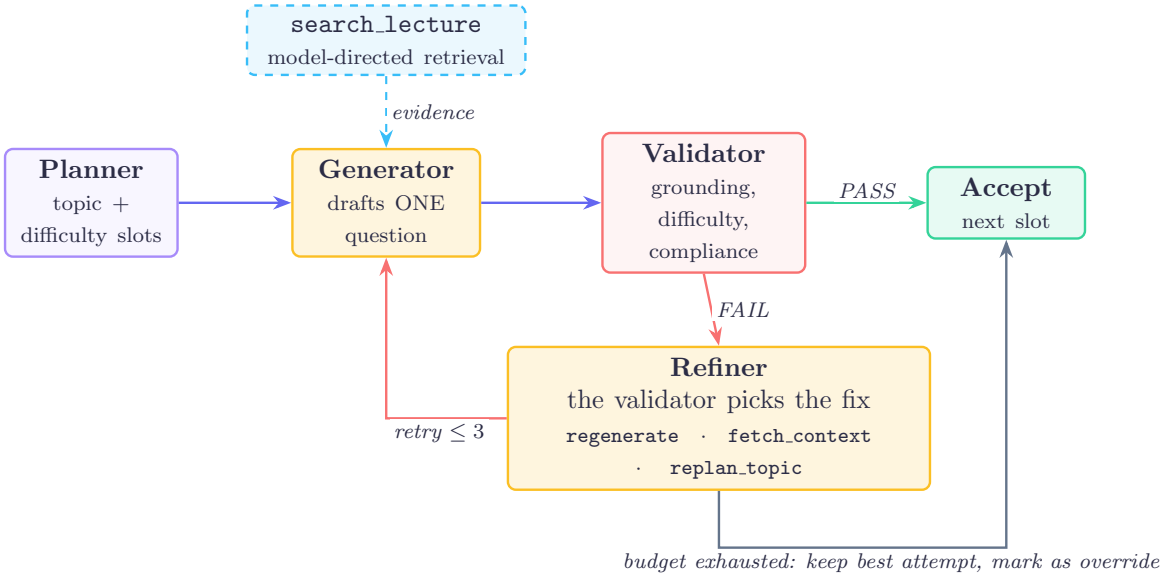

Figure 3.2: The agentic question-generation loop. Questions are produced one at a time and must pass validation before the next slot begins. The dashed edge is genuine tool use: the generator decides for itself whether it has enough grounding and issues its own retrieval query if not. On `FAIL` the validator does not merely reject — it selects the remediation, and only `regenerate` returns to the generator directly, while `fetch_context` and `replan_topic` require work outside the model first. When the retry budget is exhausted the best attempt is kept but is explicitly labelled an override, so that a failure is never recorded as a clean pass.

3.3.2 Agent contracts and failure behaviour

Every agent in the system inherits from a common base class and honours the same four-stage contract: `should_run`, which lets an agent decline work it has no input for; `plan`, which records what it intends to do; `execute`, which performs it; and `validate`, which checks its own output before the orchestrator accepts it. The contract matters more than any individual agent, because it is what makes the pipeline degrade rather than fail: an agent that raises is caught by the orchestrator, its failure is written to the agent log with the reason, and the remaining agents continue on whatever context exists. A lecture whose audio track is unreadable still produces a summary from board text alone.

Table 3.2 lists the agents, what each consumes and produces, and what happens when it cannot do its job.

Table 3.2: Agent contracts. Every agent validates its own output; the orchestrator isolates failures so that one agent cannot abort the run.

Agent	**Consumes**	**Produces**	**On failure**
Extractor	Video frames	Board and slide text, timestamped	Run continues on audio alone
Transcriber	Audio track	Transcript segments with timestamps	Run continues on board text alone
DocParser	PDF, DOCX, PPTX	Structured document text	Run aborts; there is no other source
Summarizer	Fused timeline or document text	Structured summary	Validated for length and refusal patterns, then retried
RAGBuilder	Summary and fused timeline	FAISS index over chunks	Generation falls back to the summary without retrieval
Hints	Summary and timeline	Ranked exam-focus topics	Planner falls back to summary topics
QuizPlanner	Summary, weak-topic profile	(topic, difficulty) slots	Falls back to an even spread over summary topics
Quiz generator (Easy / Medium / Hard)	One slot, retrieved context	One draft question	Retried under the loop of Section 3.3.1
QuizValidator	Draft question, accepted set	PASS/FAIL plus a remediation action	A failed parse is treated as FAIL, never as PASS
QuizRefiner	Failed draft and its reason	Revised draft	Loop falls back to plain regeneration
Answer generator	Question without the marked key	Independently derived answer	Item is flagged unverified, not dropped

Two of these entries carry the project's central design commitments. The validator treats an unparseable response as `FAIL` rather than `PASS`, so a malformed reply can never inflate the acceptance rate; and the answer generator's disagreement flags an item rather than deleting it, so that the disagreement rate remains measurable instead of being silently engineered away.

3.3.3 Reproducibility and experimental control

Because the evaluation in Chapter 4 compares two pipelines, the things held constant between them matter as much as the thing varied. This subsection records them.

Models are pinned, not chosen at run time. The generator is `gemma3:4b` and the primary model, used for summarisation, judging and grading, is `gemma3:12b`; both are open-weight and served locally through Ollama. The generator is *pinned*: when it returns unusable structured output the system retries the same model with a stricter instruction rather than escalating to the larger one. Escalation would have been the better engineering choice for output reliability, and it was deliberately rejected, because a pipeline that silently falls back to a 12B model when the 4B model struggles would no longer be measuring what it claims to measure. Every question in Chapter 4 was written by the model the chapter names.

Decoding parameters. All calls use temperature 0.2, a context window of 16,384 tokens, and an output ceiling of 8,192 tokens. The low temperature reflects the task: structured assessment items with a single defensible key, not creative text. The output ceiling is a correctness requirement rather than a tuning choice — without it the server truncates generation at a few hundred tokens, which silently cuts structured output mid-object and presents as the generator having produced nothing.

Thresholds are fixed in advance. The uniqueness check rejects a draft whose similarity to any already-accepted question exceeds 0.85; the retry budget is three attempts per slot; frame de-duplication during extraction suppresses consecutive frames above 0.97 similarity; and short-answer grading accepts a fuzzy match above 0.80. These values were set before the evaluation run and were not adjusted after seeing results.

What is *not* controlled, and why it matters. Generation is sampled rather than greedy, and no random seed is fixed, so the run is not bit-for-bit reproducible: repeating it produces a different question set with similar statistics. This is a real limitation for exact replication and is stated rather than hidden. It is mitigated by sample size, 200 questions per pipeline, and by the fact that the reported differences are consistent across three difficulty strata rather than resting on a single aggregate. Fixing a seed is a cheap improvement and is listed among the further work in Section 8.4.

The artifact trail. Each run writes the generated questions, the per-question validator verdicts with attempt counts and timings, and the full agent decision log. Measurement is performed offline on those artifacts rather than inside the generating session, so that scoring cannot influence generation and so that a scoring change can be re-applied to an existing run without regenerating it. The labelling correction disclosed in Section 4.4.1 is the one place where this separation proved insufficient, and it is the reason a confirming re-run is the first item of further work.

3.3.4 The four generation pipelines

To isolate the contribution of the agentic machinery, the same content is used to generate questions through four pipelines that differ *only* in how the model is asked:

- **Agentic** — the Plan → Retrieve → Generate → Validate → Retry loop of Section 3.3.1.
- **Non-agentic (plain)** — a single LLM call that emits the questions directly: no planning, no retrieval, no validation, no retry.
- **Chain-of-thought (CoT)** — a single call in which the model must reason step by step to *derive* each answer before committing to it, and record that reasoning.

- **Program-of-thought (PoT)** — a single call in which the model must derive each answer through an explicit, ordered sequence of computation steps (define, substitute, simplify, evaluate) and mark the option equal to the final computed value.

CoT and PoT are both *single-shot and non-agentic*: one LLM call, no planning, retrieval, validation or retry. The only thing that separates them from the plain baseline is the wording of the prompt. They emit the same JSON schema as the plain generator, plus one extra field (`reasoning` or `computation`) that is retained for inspection but ignored by the downstream judge and Answer Generator, so all four pipelines feed into an identical scoring path and are directly comparable. The two prompts are reproduced verbatim below, since the prompt *is* the method.

Placeholders in square brackets are substituted at call time: `[N]` is the requested question count, `[DIFF]` the target difficulty band (that line is omitted entirely when no band is requested), and `[CONTENT]` and `[SUMMARY]` the retrieved lecture text and its summary, truncated to 8000 and 1500 characters respectively. Line breaks are those of the prompt as sent; where a line is too long for the page it wraps with a ↪ marker.

```
Generate [N] high-quality MCQ from this lecture content.
Difficulty focus: [DIFF]
Use CHAIN-OF-THOUGHT: for EACH question, think step by step to work out the
   ↪ correct answer BEFORE choosing it, and record that step-by-step
   ↪ reasoning. The option you mark correct MUST be exactly the answer your
   ↪ reasoning reaches, and each distractor must be wrong for a stateable
   ↪ reason.
STRICT JSON: {"questions":[{"question","options":{"A","B","C","D"},"reasoning
   ↪ ","correct_answer","explanation","topic","difficulty","bloom_level","
   ↪ source_timestamp","source_type"}]}
'reasoning' = your step-by-step derivation of the correct option.

CONTENT:
[CONTENT]

SUMMARY:
[SUMMARY]
```

Prompt 3.1: Chain-of-thought generation prompt, as sent to the generator. The reasoning field is recorded for inspection and is ignored by the judge and the Answer Generator.

```
Generate [N] high-quality MCQ from this lecture content.
Difficulty focus: [DIFF]
Use PROGRAM-OF-THOUGHT: for EACH question, derive the answer through an
   ↪ explicit ordered sequence of computation/derivation steps (like a short
   ↪  program: define, substitute, simplify, evaluate), record those steps,
   ↪ and only then mark the option that EQUALS the final computed result.
   ↪ The marked correct option MUST equal the endpoint of your computation.
STRICT JSON: {"questions":[{"question","options":{"A","B","C","D"},"
   ↪ computation","correct_answer","explanation","topic","difficulty","
   ↪ bloom_level","source_timestamp","source_type"}]}
'computation' = your ordered derivation steps ending at the correct value.

CONTENT:
[CONTENT]

SUMMARY:
[SUMMARY]
```

Prompt 3.2: Program-of-thought generation prompt. Identical to Prompt 3.1 except for the reasoning instruction and the name of the recorded field, so the two differ only in how the answer is required to be derived.

3.3.5 Answer correctness: two agents that must agree

A question can be well written and still mark the wrong option. Question generation and answer generation are therefore treated as *two distinct agents*, and their answers are cross-checked. Correctness is measured three complementary ways:

- **Independent re-solve (Answer Generator agent).** The larger model is given the stem and the options *with the marked answer removed*, and solves the question from scratch. Its choice is compared with the Question Generator's marked answer; the agreement rate is the *independent-match rate*. The solver never sees what it is supposed to agree with.
- **Judge-verify.** While scoring quality, the judge is additionally asked whether the marked answer is actually correct given the content, yielding the *judge-verified-correct rate*. This is free — it rides along on a call that was already being made.
- **Calibration on a known-answer set.** The generated questions have no ground-truth key, so neither of the above is an absolute measure of correctness — if generator and solver share a misconception, they agree and are both wrong. To bound this, both models independently solve a set of MCQ whose answers *are* known, and their accuracy on it calibrates how much the agreement rate above is worth. The calibration set is stratified into *computational* items, whose answers are computed with

SymPy [31] rather than typed by hand, and *conceptual* items drawn from the chapter's definitions and theorems. This is reported separately and is explicitly *not* a score on the generated questions.

The calibration set is used for evaluation only. It is never placed on the generation path, since a generator that had seen the answer key would be scored on questions it had effectively been given, and the resulting numbers would be contaminated.

3.3.6 System snapshot

Figures 3.3 and 3.4 show the deployed system in use. They are included because the agentic loop described in Section 3.3.1 is not only an internal mechanism: it is surfaced directly to the user, and one of the design commitments of this project is that a student should be able to see how a question was produced rather than being handed an opaque result.

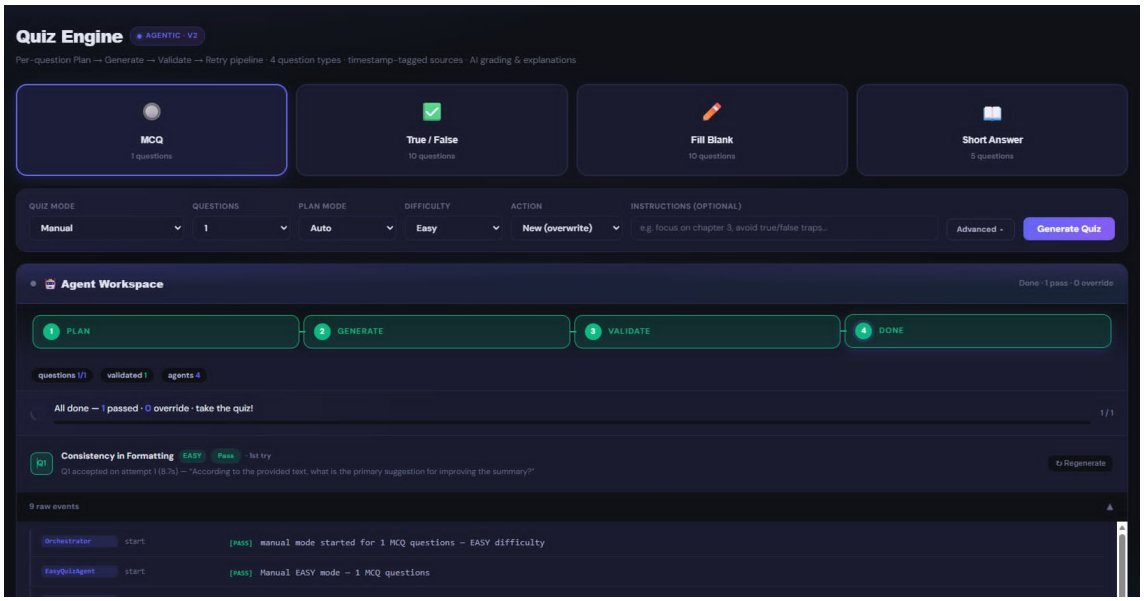

Figure 3.3: The Quiz Engine and its live Agent Workspace. The four stage indicators across the top track the pipeline in real time as it runs. Beneath them, the counters report questions completed, questions validated and agents active, and each generated question appears as its own row recording the topic, the target difficulty, the verdict, the attempt on which it was accepted and the time taken. The row shown here was accepted on the first attempt in 8.7 seconds. The raw event log at the bottom names the agent responsible for every step, so a run can be audited after the fact rather than inferred from its output.

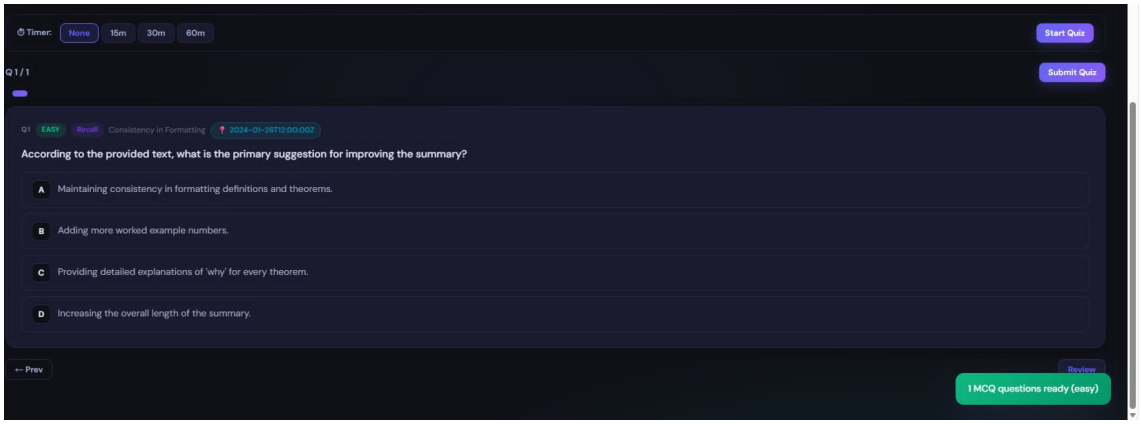

Figure 3.4: The quiz-taking view. Each item carries its target difficulty, its Bloom level and the topic it was planned against, together with the source timestamp of the lecture material it was grounded in. That timestamp is what allows a student who doubts an answer to return to the exact moment in the lecture the question came from, which is the practical safeguard against the failure mode discussed in Chapter 5.

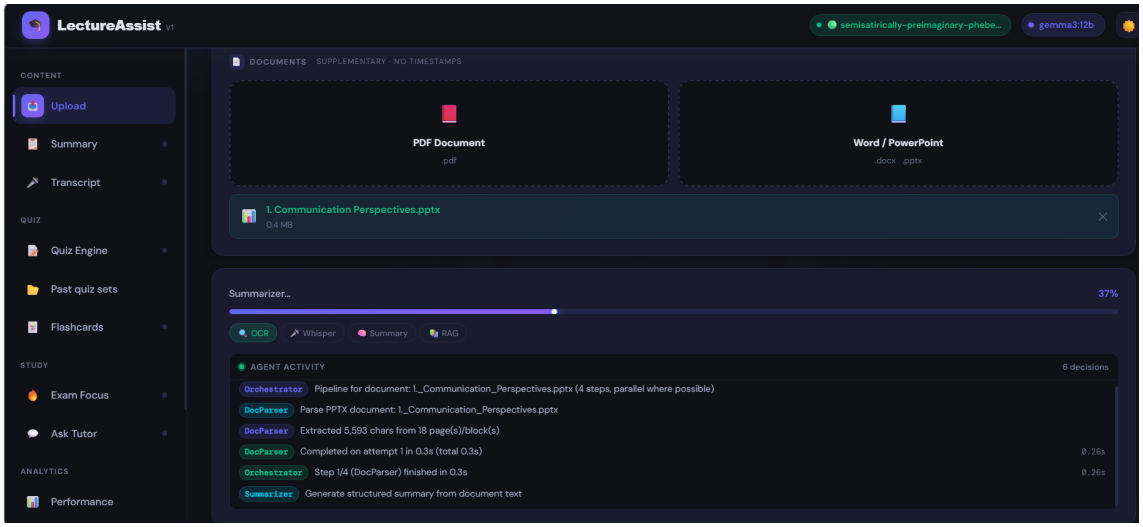

Figure 3.5: Processing in progress. The agent activity feed streams each orchestrator decision as it is taken rather than reporting them only once the run has finished: the capture shows six decisions logged at 37% completion, with the DocParser having extracted 5,593 characters from 18 blocks in 0.3 seconds and the Summarizer already started. The stage pills above it track which of OCR, Whisper, Summary and RAG is currently active.

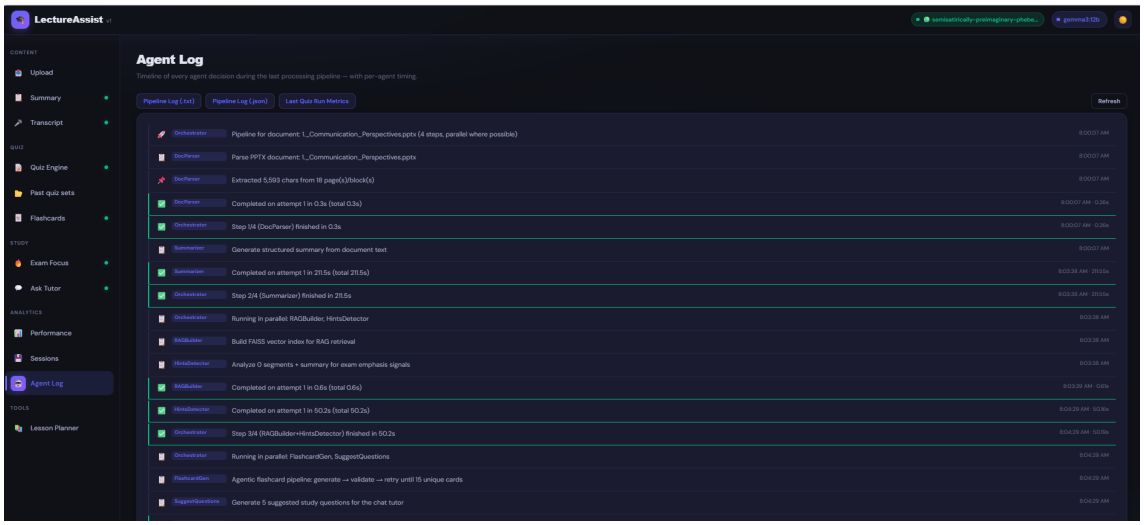

Figure 3.6: The agent log for a completed document run, with per-agent timing. The trail records not only which agent acted but how long each took and which steps the orchestrator chose to run concurrently: DocParser completed in 0.3s, the Summarizer in 211.5s, and RAGBuilder and HintsDetector were dispatched in parallel, finishing in 0.6s and 50.2s respectively. The pipeline is therefore auditable after the fact, which is what makes the timing claims in this chapter checkable rather than asserted.

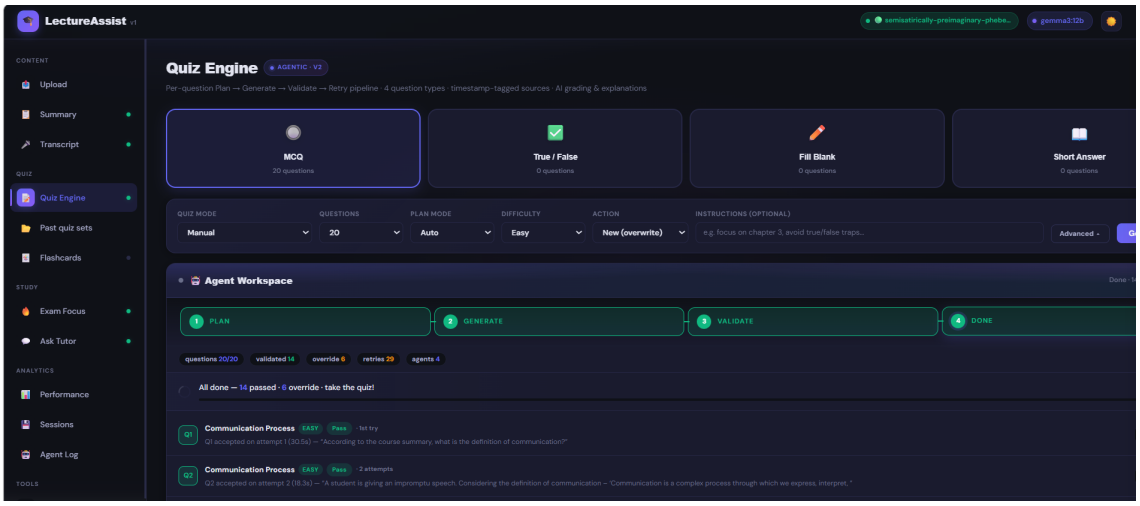

Figure 3.7: A full twenty-question generation. The counters read *questions* 20/20, *validated* 14, *override* 6, *retries* 29, *agents* 4, and the status line states plainly that 14 passed and 6 were kept as overrides. Twenty accepted questions therefore cost twenty-nine retries, which is the compute multiplication discussed in Section 5.1 made visible.

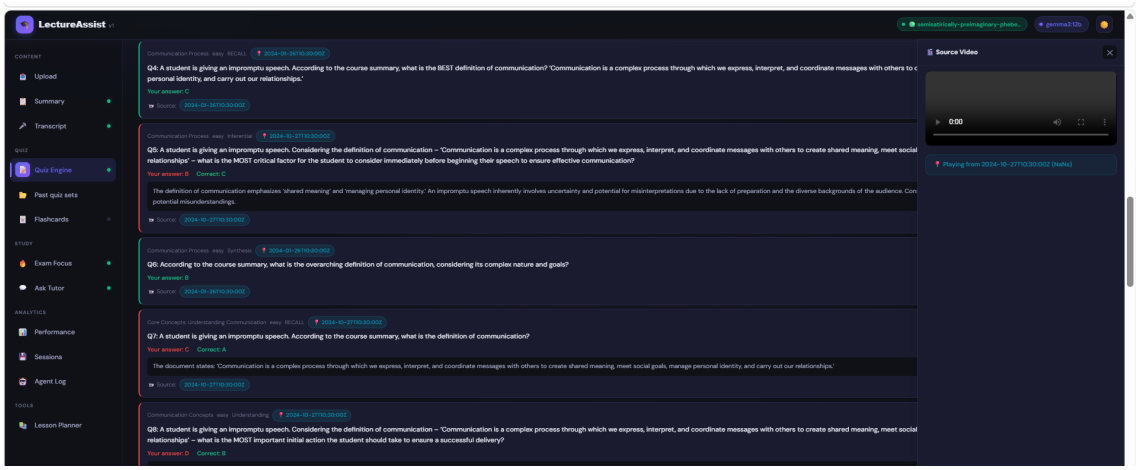

Figure 3.8: The graded review, with the source panel open. Each item shows the marked answer against the student's, the explanation drawn from the retrieved material, and the lecture timestamp the question was grounded in; selecting that timestamp opens the source at that moment. Items answered incorrectly are separated from those answered correctly, so the review surface is also the input to the per-topic weakness profile of Section 3.3.7.

The counters in Figures 3.3 and 3.7 distinguish questions that passed validation from those kept as overrides, and Figure 3.7 is the more informative of the two because it is a run in which overrides actually occurred: six of its twenty items exhausted the retry budget. This distinction is shown rather than hidden for the reason given in Section 3.3.1: a pipeline that silently recorded an exhausted retry budget as a clean pass would report an acceptance rate that was a fiction, and the interface is built to make that impossible. The same figure is also the honest counterweight to the loop's benefits, since it shows what the loop costs.

3.3.7 Multi-tier grading and adaptation

Student answers are graded by a cascade chosen to match the question type. MCQ and True/False are graded by exact key match. Fill-in-the-Blank is graded by a fallback chain of exact match, then containment, then fuzzy string similarity above a 0.80 threshold, so that a correct answer is not marked wrong over a typo or a plural. Short Answer, where no string comparison is adequate, is graded by the 12B model against a rubric, which also returns concept-level feedback naming what the student missed rather than merely marking it incorrect.

Every graded response updates a persistent per-topic profile. The Adaptive Quiz agent reads that profile and biases the next quiz's topic and difficulty allocation toward the topics where the student's accuracy is lowest, so that practice concentrates where it is needed instead of being uniformly distributed.

Chapter 4

Investigation/Experiment, Result, Analysis and Discussion

This chapter reports the evaluation of LectureAssist's question generator. It describes the environment the experiment was run in, enumerates the configurations under test, defines the evaluation instruments precisely, and reports the measured results together with an analysis of what they establish and what they do not.

The evaluation has two halves. The first asks whether the agentic pipeline produces better multiple-choice questions than single-shot prompting, measured with a suite of automated metrics native to this project and defined in Section 4.2.1. Those metrics combine four judged quality dimensions with a set of corpus-level measures computed directly from the generated questions, and they are what carries every result claimed in this chapter.

The second half follows directly from the first half's outcome. Because the measurements show the agentic pipeline *beating* its own baseline on every metric that discriminates (Section 4.4.1), the next question is no longer "does the agentic loop win" but "*which agent* is responsible, and how much does each one cost". A five-agent pipeline that wins as a whole tells us nothing about whether all five agents are needed, and the pipeline is roughly an order of magnitude more expensive than the single call it beats; an advantage that survives deleting an agent is an advantage that agent was not producing. Section 4.3 sets out the controlled design that isolates this, in two families: **leave-one-agent-out** variants, which delete one agent at a time (Section 4.3.1), and **agent-merging** variants, which collapse two agents into a single LLM call (Section 4.3.2). That design is a contribution of this chapter in its own right, because it fixes in advance how each outcome is to be read and therefore removes the freedom to rationalise a result after seeing it.

The experiment is designed around five questions. *RQ1:* does the agentic pipeline produce better multiple-choice questions than single-shot prompting? *RQ2:* how does each pipeline behave as the target difficulty rises from easy to hard? *RQ3:* which dimensions actually separate the pipelines, and which saturate? *RQ4:* which individual agent accounts for the agentic pipeline's behaviour, in particular, is the retry loop the source of its diversity

advantage? *RQ5:* can two agents be merged into one call without losing quality, and how much compute does that save?

RQ1 to RQ3 are answered by measurement in Section 4.4. RQ4 and RQ5 are questions the measurements raise but cannot settle, and Section 4.3 specifies the experiments that would.

4.1 Environment Setup

Question *generation* was run on a single Google Colab instance with an NVIDIA T4 GPU (16 GB), with models served locally by Ollama; no paid or proprietary API is used anywhere in the generation path. Bulk generation artifacts are written to Google Drive so a run survives a Colab disconnection and can be resumed rather than restarted.

4.1.1 Configurations under test

Eleven configurations are defined. All share the same pinned `gemma3:4b` generator, the same source material, the same retrieval tool and the same cognitive-routing Controller; they differ only in which agents are present and how those agents are packaged into LLM calls. Holding everything else fixed is what makes any difference attributable to the manipulation rather than to a confound.

Table 4.1: The eleven configurations defined for this study and the question each one is designed to answer.

Configuration	Manipulation	Question it answers
Baseline comparison (RQ1–RQ3)		
`agentic`	full loop: Plan → Retrieve → Generate → Validate → Retry	control
`nonagentic`	one LLM call, no planning, retrieval, validation or retry	does the machinery help?
`cot`	single call, chain-of-thought prompt	can a prompt substitute?
`pot`	single call, program-of-thought prompt	can a prompt substitute?
Leave-one-agent-out ablations (RQ4)		
− planner	uniform slots at the target difficulty; no Quiz Planner	does planning help, or cause topic tunnel-vision?
− retrieval	fixed content snippet; `search_lecture` disabled	is model-directed RAG earning its cost?
− validator	first draft accepted; no validation, no retry	the whole quality gate
− refiner	validator still runs and labels, but never remediates	**separates detection from remediation**
− routing	`COGNITIVE_ROUTING=False`	does the Controller buy diversity?
Agent-merging variants (RQ5)		
`merge_vr`	Validator + Refiner in ONE call: it judges the draft and returns the rewrite in the same reply	is a separate refiner call worth it?
`merge_pg`	Planner + Generator in ONE call: the generator picks its own topic and drafts the question	is a separate planner call worth it?

The baseline pipelines read the document through an identical sliding window that

steps across the whole chapter with a uniform batch size, so no baseline is advantaged by seeing more or different content, and any difference between them reflects the prompting strategy alone.

Source data. Questions are generated from a college-level STEM document, the *Limits* chapter of OpenStax *Calculus, Volume 1*, ingested through the standard document pipeline (extraction → map-reduce summary → retrieval index). The extracted chapter text (about 112k characters) is also supplied to the judge as the ground-truth reference against which each question is scored.

Question count. Each of the two evaluated pipelines generated 200 MCQs spread across three target difficulties (easy n=68, medium n=66, hard n=66). The ablation and merging configurations are specified at n=100 per difficulty rather than 200: duplicate rate and grounding are large effects and separate at that size, and eleven configurations at 200 each is not affordable on a free-tier T4.

4.2 Evaluation Protocols

Generation and measurement are kept strictly separate: the Colab run only *generates* and saves every question to a single JSON artifact; standalone harnesses then read that artifact and score each question. Scoring an already-generated corpus with a separate program means the measurement can be repeated, or redone under a different protocol, without touching the system under test. It also means the ablation and merging corpora feed into an *identical* scoring path: the harnesses key on a `pipeline:difficulty` group label and are indifferent to which configuration produced a question.

Every measure is reported on *its own native scale*, and no score is rescaled onto another's range. A judged dimension on a five-point scale and a rate in $[0, 1]$ encode different things, and a linear remapping between them would produce a number that means nothing, so the two groups are always tabulated separately.

4.2.1 Native metrics, and why a cost column is mandatory

This project reports its own automated metrics: four judged quality dimensions on a 1–5 scale (`gemma3:12b` judging a `gemma3:4b` generator), plus automated measures in $[0, 1]$, acceptance rate, judge-verified correctness, independent answer match, semantic grounding, duplicate rate, effective acceptance rate and inter-question similarity.

These are joined by a **cost** measure: the number of LLM calls consumed per accepted question, recorded per question at generation time. This is not decoration. An ablation result of the form "removing agent X barely changed quality" is uninterpretable on its own; it becomes a finding only when read next to "and it removed a third of the calls". Equally, a merging result that preserves quality is worthless if it does not actually save compute. No configuration is declared preferable on quality alone.

4.3 Ablation Design

4.3.1 Leave-one-agent-out

The agentic pipeline is not a monolith; it is five separable parts: the Quiz Planner, the model-directed retrieval tool, the Validator, the Refiner, and the cognitive-routing Controller. The leave-one-out family deletes exactly one of them per configuration, holding the other four and every hyper-parameter fixed (Table 4.1).

Two of these configurations deserve emphasis because they form a matched pair. The − validator condition removes the quality gate entirely: a draft is accepted as written. The − refiner condition keeps the Validator, and keeps its verdict, but forbids it from acting: a failing question is labelled and retained rather than regenerated. Together they separate *detection* from *remediation*: the first asks whether noticing a bad question helps at all, the second whether acting on that verdict helps.

That pair is the sharpest instrument in this chapter, because it tests a specific mechanism this work has already proposed but never verified. Section 4.4.1 finds the agentic pipeline producing substantially *fewer* near-duplicate questions than the plain baseline, and Section 4.4.1 attributes that to the retry loop: a draft that repeats an accepted question is rejected, and the retry is given the failure reason, so the loop suppresses duplication rather than resampling into it. If that explanation is correct, then − refiner, which retains validation but never acts on its verdict, should lose most of the diversity advantage and show a duplicate rate close to the baseline's, while − validator, which removes detection entirely, should lose all of it. If instead − refiner retains the advantage, then detection alone is not what produces it and the credit belongs elsewhere, most plausibly to the planner distributing slots across topics before any question is drafted. Either outcome is informative; at present the explanation is an untested hypothesis and is labelled as such.

4.3.2 Agent merging

Where the leave-one-out family asks *whether* an agent is needed, the merging family asks whether two agents need to be two *LLM calls*. The distinction matters because the agentic pipeline's cost is dominated by call count, not by any single agent's presence.

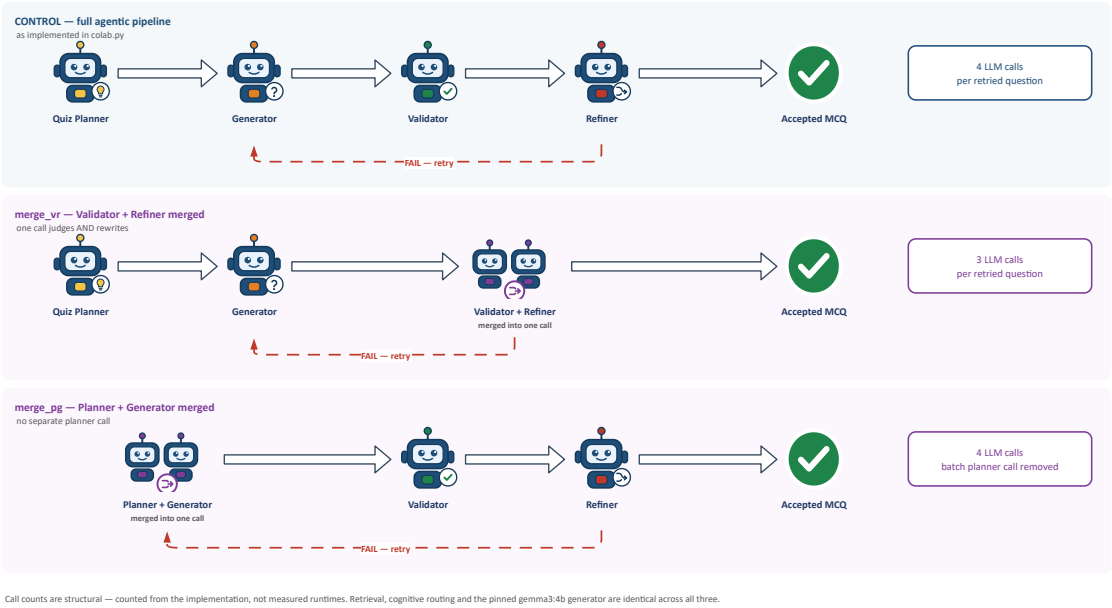

Figure 4.1: The three configurations compared by the merging experiment. All three receive identical grounded content and retrieval index and share the pinned `gemma3:4b` generator, the retrieval tool and the Controller; only the boxed agent merges differ. Call counts are structural, counted from the implementation, not measured runtimes.

`merge_vr`, Validator + Refiner. In the control pipeline a failing question costs two calls to fix: one to validate, and a second to redraft with the failure reason fed back. In `merge_vr` the validator is instructed to repair what it rejects and to return the corrected question in the same reply, so a rejected question is re-validated directly rather than redrafted first.

Only the `regenerate` remediation can be absorbed this way. The other two actions the validator may choose `fetch_context` and `replan_topic`, require work *outside* the model (a retrieval query, a topic swap) and are still returned as actions for the loop to act on, exactly as in the control. A merge cannot remove a step that was never an LLM call.

`merge_pg`, Planner + Generator. The control spends one planner call per batch and then one drafting call per question against an assigned topic. In `merge_pg` there is no standalone planner: each drafting call is shown the topics already used and must itself choose an unused topic that the summary genuinely covers, then write the question for it in the same reply. This trades a global view for a local one: the model can no longer balance topic counts across the whole quiz, only avoid what it has already seen. Whether that costs diversity is precisely what the experiment measures.

upper bound rather than a prediction.

4.3.3 What these ablations cannot show

Three limitations are inherent to the design and are stated here rather than discovered later. First, leave-one-out attributes an effect to the *absence* of one agent in the presence of the other four; it cannot identify interactions between agents, for which a factorial design far beyond this project's compute budget would be required. Second, every quality number is produced by an LLM judge from the same model family as the generator, so an ablation that changes quality only slightly may be changing something the judge cannot see. Third, all configurations are evaluated on one chapter of one subject, so an agent that appears unnecessary on calculus may be necessary on material with different structure.

4.4 Results and Discussion

4.4.1 Native metrics, agentic versus non-agentic

Table 4.2 reports this project's own automated metrics for the two configurations that have completed a scored run. These are on a five-point scale for the judged quality dimensions and on a 0–1 rate scale for the automated measures; they are reported separately from both published protocols and are never mixed with them. The judge is `gemma3:12b`, generating from a `gemma3:4b` generator.

Table 4.2: Native metrics, n=200 questions per pipeline. Judged dimensions are on a 1–5 scale; the remaining measures are rates or cosine scores in $[0, 1]$. Higher is better for every row *except* duplicate rate and inter-question similarity.

Metric	Scale	Non-Agentic	Agentic
Overall judged quality	1–5	4.86	**4.93**
Correctness	1–5	4.98	**5.00**
Clarity	1–5	4.92	**4.97**
Difficulty match	1–5	4.36	**4.39**
Acceptance rate	0–1	0.995	**1.000**
Judge-verified correctness	0–1	0.995	0.995
Independent answer match	0–1	0.940	**0.970**
Semantic grounding	0–1	0.557	**0.624**
Duplicate rate (↓)	0–1	0.385	**0.280**
Effective acceptance rate	0–1	0.612	**0.720**
Inter-question similarity (↓)	0–1	0.932	**0.894**

Table 4.3: Native metrics broken down by target difficulty, n per cell as shown. Higher is better except duplicate rate.

Difficulty	Pipeline	n	Duplicate	Grounding	Eff. accept	Indep. match	Verified
Easy	Non-agentic	68	0.353	0.562	0.647	0.971	1.000
	Agentic	68	**0.324**	**0.642**	**0.676**	0.971	0.985
Medium	Non-agentic	66	0.379	0.595	0.612	0.879	0.985
	Agentic	66	**0.227**	**0.614**	**0.773**	**0.955**	1.000
Hard	Non-agentic	66	0.288	0.513	0.712	0.970	1.000
	Agentic	66	**0.182**	**0.616**	**0.818**	**0.985**	1.000

Why the pooled rates in Table 4.2 exceed every per-difficulty rate in Table 4.3. The duplicate rate is not a per-question property that can be averaged; it is a property of a *corpus*, computed by comparing every question against every other. Pooling the three difficulty groups into one corpus of 200 therefore creates cross-difficulty collision opportunities that do not exist inside any single group, and the pooled rate is correspondingly higher than the weighted mean of the three group rates (0.280 against 0.245 for the agentic pipeline, 0.385 against 0.340 for the baseline). Semantic grounding, by contrast *is* a per-question mean, and its pooled values reproduce the weighted means exactly (0.624 and 0.557), which is a useful internal consistency check on the two tables. Effective acceptance is defined as

$$\text{EffAccept} \;=\; \text{AcceptRate} \times \big(1 - \text{DuplicateRate}\big) \tag{4.1}$$

so it inherits the pooling behaviour of the duplicate rate: with an acceptance rate of 1.000 the agentic pipeline's pooled effective acceptance is exactly $1 - 0.280 = 0.720$, and with an acceptance rate of 0.995 the baseline's is $0.995 \times (1 - 0.385) = 0.612$. Every effective-acceptance figure in both tables satisfies Equation 4.1.

A declared correction to the pipeline labelling. The two arms were interchanged at generation time: the group recorded as `agentic` in the source artifact was in fact produced by the non-agentic pipeline, and vice versa. Both tables above apply the corrected labelling, and it is the corrected labelling that every statement in this chapter refers to. This is disclosed rather than quietly applied because the script that assigned the original labels is no longer in the repository, so the assignment cannot be re-derived from the artifacts alone; the correction rests on the generation records rather than on the JSON keys. A confirming re-run is the first item of future work in Section 8.4.

Reading these results honestly. On this instrument the agentic pipeline **outperforms** the non-agentic baseline. It matches or beats the baseline on every judged dimension, produces markedly fewer duplicates (0.280 versus 0.385), achieves a higher effective acceptance rate (0.720 versus 0.612), is better grounded in the source text (0.624 versus 0.557), and agrees more often with an independent solver (0.970 versus 0.940). The pattern holds at every difficulty level, and the margin *widens* as difficulty rises: on hard questions the agentic pipeline halves the duplicate rate (0.182 versus 0.288) and leads on grounding by more than ten points (0.616 versus 0.513).

Three observations follow, and the first is a warning about the second.

First, the four judged quality dimensions *saturate*. Every value sits between 4.36 and 5.00 on a five-point scale, and correctness reaches the ceiling exactly. A metric that cannot go up cannot discriminate, and this saturation is a known failure mode of judged quality scales in the educational question-generation literature reviewed in Chapter 2, not a peculiarity of this judge. The agentic pipeline's nominal wins on those four rows, 4.93 against 4.86, 5.00 against 4.98, are therefore close to meaningless, and no weight is placed

on them here. **The agentic advantage claimed in this section rests entirely on the automated metrics**, which have room to move and which separate the two pipelines by margins an order of magnitude larger than the judged rows do.

Second, the metrics that do discriminate, duplicate rate, semantic grounding, effective acceptance and independent answer match, favour the agentic pipeline consistently, at every difficulty, without a single reversal. That consistency is what makes the result credible despite the saturation above: four independent measures computed in different ways, over three difficulty strata, all point the same direction.

Third, the plausible mechanism is the validate-and-retry behaviour itself. A draft that repeats an already-accepted question or is weakly grounded in the source is rejected, and the retry is fed the specific failure reason rather than merely resampled, so the loop actively suppresses duplication and pulls generation back toward the source material. The grounding and duplicate results are exactly what that mechanism predicts. **This nevertheless remains a hypothesis.** It is consistent with the data but is not established by it: the measurement shows *that* the full pipeline wins, not *which* of its five agents produces the win. Separating those is what the leave-one-agent-out design of Section 4.3.1 is built to do, and on the present evidence the attribution to the retry loop remains an inference rather than a finding.

4.5 System Interpretation

The previous section established *what* the measurements show. This section asks what they imply about the system that produced them: which parts of the architecture the numbers can speak to, which they cannot, and what the shape of the result says about the design. Everything here is interpretation and is labelled as such; the evidential claims remain those of Section 4.4.1.

4.5.1 Where in the pipeline the advantage can live

The four metrics that separate the pipelines are not independent of one another, and their pattern is informative. Duplicate rate and inter-question similarity are *corpus-level* properties: they can only be affected by a component that has some view of what has already been produced. Semantic grounding is a *context-level* property: it can only be affected by a component that changes what source material reaches the generator. Independent answer match is an *item-level* property: it depends on whether a single question and its marked key are mutually consistent.

The single-shot baseline has no mechanism that acts on any of the three. It sees a window of source text and emits questions; nothing in it compares a draft against earlier drafts, retrieves different material in response to a weak draft, or re-examines an item before accepting it. The agentic pipeline has exactly one mechanism acting on each: the validator's uniqueness check for the corpus level, the retriever together with

the `fetch_context` remediation for the context level, and the validate-and-retry cycle for the item level. The advantage therefore appears on precisely the three properties the architecture adds machinery for, and on no others. That correspondence is the strongest structural argument available from this data that the gain comes from the loop rather than from incidental differences between the runs.

It is an argument from design rather than from measurement, and it should be read at that strength. A single run cannot exclude the possibility that some uncontrolled difference produced the same pattern by coincidence, which is why the leave-one-agent-out design of Section 4.3.1 exists and why the attribution is not claimed as a finding.

4.5.2 Why the margin widens with difficulty

The most interpretable feature of Table 4.3 is that the gap is smallest on easy questions and largest on hard ones. On duplicate rate the two pipelines are nearly level at easy (0.324 against 0.353) and separated by a factor of more than one and a half at hard (0.182 against 0.288); grounding follows the same pattern, with the baseline degrading from 0.562 to 0.513 as difficulty rises while the agentic pipeline improves from 0.642 to 0.616.

A plausible reading is that difficulty and the failure modes interact. An easy question can be generated adequately from almost any passage of a chapter, so the space of acceptable outputs is large, retrieval quality matters little, and two questions drawn from that space are unlikely to collide. A hard question must combine or apply specific material, so the space of acceptable outputs is much smaller: retrieval quality matters a great deal, and independent attempts are far more likely to converge on the same few tractable topics. On this reading the baseline does not get worse at hard questions so much as the hard setting exposes weaknesses that the easy setting hides, and the loop's contribution is largest exactly where the unaided generator has least room to succeed by chance.

This is the practically important form of the result, because the questions worth generating automatically are the ones a student cannot easily write for themselves.

4.5.3 What the saturation implies for system design

The four judged dimensions sitting between 4.36 and 5.00 is a result about the instrument, but it has a design consequence. If judged quality cannot distinguish a five-agent pipeline from a single call, then it also cannot serve as the signal a system uses to decide whether a question is good enough to keep. A validator that asked a language model to score clarity on a five-point scale and accepted anything above a threshold would accept essentially everything, and the acceptance rates of 1.000 and 0.995 in Table 4.2 show that this is close to what happens.

The validator in this system is therefore not built as a quality scorer. It tests specific, falsifiable properties: whether the question is answerable from the retrieved context, whether it matches the requested difficulty band, and whether it duplicates an already-accepted item. Those checks discriminate because they can fail. This is the transferable

design lesson of the chapter, and it is independent of whether the agentic advantage survives the ablations: **a validation gate must test properties that can be false, not qualities that can be rated**.

4.5.4 What the system does not establish

Three limits bound every interpretation above. The measurement compares a whole pipeline against a single call, so it cannot apportion credit among the five agents. The judge and the independent solver share a model family with the generator, so their agreement measures consistency rather than truth, and the absolute rates should be read as a ranking signal between pipelines rather than as a rate of correctness. And the entire result rests on one chapter of one English mathematics textbook, so nothing here supports a claim about other subjects, other languages, or lecture video as opposed to document input.

The pipeline labelling correction recorded in Section 4.4.1 compounds the third limit rather than adding a separate one: it rests on the generation records rather than on the stored artifacts, and a confirming re-run is the cheapest way to remove that dependency. Until that re-run exists, the interpretation in this section should be read as the most coherent account of the data available, not as a settled explanation.

4.6 Summary

This chapter compared the agentic question-generation pipeline against a single-shot baseline on 200 multiple-choice questions per pipeline, generated from one chapter of a college calculus textbook and spread evenly across three target difficulties. Both pipelines used the same pinned `gemma3:4b` generator, read the source document through an identical sliding window, and were scored through an identical measurement path, so that any difference between them is attributable to the generation strategy alone.

What was established. The agentic pipeline outperforms the baseline on every measure capable of separating them. It produces fewer near-duplicate questions (0.280 against 0.385), grounds its questions more firmly in the source text (0.624 against 0.557), achieves a higher effective acceptance rate (0.720 against 0.612), and agrees more often with an independent solver that never sees the marked option (0.970 against 0.940). The result holds at easy, medium and hard difficulty without a single reversal, and the margin grows with difficulty: on hard questions the agentic loop roughly halves the duplicate rate and leads on grounding by more than ten points. Four measures, computed in four different ways, over three difficulty strata, all point the same way, and it is that consistency rather than the size of any individual gap that makes the conclusion credible. This answers RQ1 and RQ2.

What the instruments revealed about themselves. The four judged quality dimensions saturate. Every value falls between 4.36 and 5.00 on a five-point scale, and correctness reaches the ceiling exactly, so the differences on those rows are too small to carry weight and none is claimed. The measures that discriminate are the corpus-level ones: duplication, grounding, effective acceptance and independent answer match. This answers RQ3, and it is the most transferable finding in the chapter, because it implies that any evaluation of automatic question generation reporting judged quality alone will conclude that all reasonable systems are excellent and that none can be told apart. The signal in this study lives entirely in measures that a study of that design would never have computed.

What remains open. The measurement establishes that the five-agent pipeline beats one LLM call. It does not establish which of the five agents produces that advantage, and the pipeline costs roughly an order of magnitude more compute than the call it beats, so the attribution matters both scientifically and practically. The mechanism proposed in Section 4.4.1, that validation and reasoned retry suppress duplication and pull generation back toward the source, is consistent with every number reported here and is established by none of them. It is labelled a hypothesis throughout for that reason. The leave-one-agent-out and agent-merging designs specified in Section 4.3 exist to settle it, and their reading rules are fixed in advance so that the answer cannot be chosen after the fact. RQ4 and RQ5 are therefore posed and designed for here rather than answered here, and Chapter 8 carries them forward as the first items of further work.

Two caveats carried forward. The judge and the independent solver are both language models drawn from the same family as the generator, so agreement between them is evidence of consistency rather than proof of correctness, and absolute rates should be read as a ranking signal between pipelines rather than as a measure of how often the system is right. Separately, the pipeline labelling correction disclosed in Section 4.4.1 rests on the generation records rather than on the stored corpus, and a confirming re-run is the cheapest and most important piece of further work this chapter identifies.

Chapter 5

Impacts and Design Constraints of the Project

This chapter examines the non-technical constraints that shaped LectureAssist, economic, social, legal, ethical and environmental, and the impact the system is likely to have once deployed. Several of the most consequential engineering decisions in this project, notably the refusal to depend on any paid cloud API, were driven by these constraints rather than by technical considerations. They are therefore not an afterthought appended to a finished design; they are among the inputs that produced it.

5.1 Economic Impact

The constraint that drove the architecture. The system was required to run without a paid API. This is not a stylistic preference, and it is worth being precise about why. Per-token cloud pricing makes routine, high-volume practice, which is precisely the usage pattern that makes retrieval practice effective, prohibitively expensive for students in low- and middle-income settings. It also makes the tool's availability contingent on a credit card and an international payment rail that many students in Bangladesh and comparable settings simply do not have; the barrier is not only the price but the mechanism of payment. Every model in LectureAssist is therefore open-weight and locally served. The consequence is a hard VRAM budget, which is why the generator is a 4B model rather than a frontier one, and much of the engineering documented in Chapter 3 exists to extract reliable structured output from a small model. The evaluation in Chapter 4 can be read as a test of whether that constraint is survivable: it finds that a well-structured agentic loop around a 4B model measurably outperforms a single call to the same model, which is the mechanism by which the cost constraint is absorbed rather than merely accepted.

Where the cost actually goes. Removing the per-token bill does not make the system free; it moves the cost from a recurring charge to a one-off hardware requirement, and it is worth being clear about the trade. A cloud deployment charges per question generated,

so cost scales with exactly the behaviour the system is trying to encourage: a student who practises more pays more. Local inference inverts that. The marginal cost of the thousandth question is the electricity to generate it, which means the usage pattern that makes retrieval practice effective is the pattern the cost model rewards. What replaces the bill is a floor: the system needs a GPU with enough memory to hold a small generator and a larger judge, which in this project is met by the free tier of a hosted notebook rather than by hardware the student owns.

The compute cost of the agentic loop. The architecture is not free of economic consequence either. As Chapter 4 records, the five-agent pipeline costs roughly an order of magnitude more compute than the single call it outperforms, because planning, retrieval, validation and retry each add model invocations. On a paid API that multiplication is what makes multi-agent decomposition expensive, and it is why the surveyed systems that adopt it also depend on commercial inference. On local inference the same multiplication costs time rather than money, which is the specific reason the design is affordable here and would not be affordable in the cloud. The ablation designs of Section 4.3 exist partly to establish whether the whole multiplication is necessary or whether a cheaper subset of the agents captures most of the benefit, which is an economic question as much as a scientific one.

Institutional adoption. For a department rather than an individual, the relevant comparison is against the staff time currently spent writing practice items by hand. The system does not replace that work, because its output is unreviewed until a human reviews it, but it changes the task from authoring to checking. Whether that is a saving depends on how much of the generated set survives review, and this project measures acceptance under an automated judge rather than under an instructor, so the honest position is that the institutional case is plausible but not evidenced here.

5.2 Impact on Societal, Health, Safety, Legal and Cultural Issues

Educational and societal impact. The core societal argument for LectureAssist is one of access. Retrieval practice, repeatedly testing oneself on material rather than re-reading it, is among the most effective study strategies known, and the effect is large, replicated and long established [32, 33]. In the most comprehensive review of study techniques available, practice testing is one of only two techniques rated as having high utility across learners, materials and criterion tasks [34]. Yet a supply of good practice questions is a resource distributed very unevenly. A student at a well-resourced institution has question banks, teaching assistants and instructor office hours; a student without those has the end-of-chapter exercises and nothing else, and those exercises are not tied to what their particular instructor actually emphasised in class. By generating unlimited, validated,

difficulty-controlled questions from the student's *own* lecture material, the system puts a form of practice that was previously a privilege of well-resourced institutions within reach of anyone with a laptop. This aligns directly with UN Sustainable Development Goal 4 (Quality Education) and, because it removes a cost barrier rather than an ability barrier, with Goal 10 (Reduced Inequalities).

Safety: the risk of confidently teaching the wrong thing. The most serious safety issue in a system of this kind is not a crash: it is a fluent, well-formed multiple-choice question whose marked answer is *wrong*. A student who trusts the tool will learn the error, rehearse it through exactly the retrieval practice that makes the tool valuable, and carry it into an exam. The tool's fluency works actively against their catching it, because the surface features a student uses to judge whether a question is trustworthy, clear wording, plausible distractors, a confident explanation, are precisely the features a language model produces most reliably, whether or not the marked option is correct.

This risk shaped the evaluation methodology rather than merely being noted in it. Answer correctness is measured separately from question quality, by an Answer Generator agent that re-solves each question without ever seeing the marked option (Section 3.3.5), and the solver is itself calibrated against questions whose answers are known independently, so that its verdict is not over-trusted. The honest conclusion is that the system's output is suitable for *practice* and must not be treated as an authoritative answer key. Any deployment should surface this to the student directly, and no generated question should be used for graded assessment without human review. The design implication is concrete: the interface shows the source timestamp for every question, so that a student who doubts an answer can jump to the moment in the lecture it came from rather than having to trust or discard the question wholesale.

Legal and privacy considerations. Lecture recordings and course materials are frequently copyrighted by the instructor or the institution, and lecture audio contains the identifiable voices of the instructor and, incidentally, of students who ask questions. Uploading such material to a third-party cloud service to be processed, and potentially retained, logged, or used for training, raises copyright and data-protection questions that most students are not in a position to evaluate, and that they are typically not even prompted to consider. Because LectureAssist runs entirely locally, the lecture never leaves the machine the user controls: there is no third-party processor, and therefore no third-party retention. Users must still hold the right to use the material they upload, and the system should not be used to redistribute copyrighted lectures. The persistent per-topic performance profile is educational data about an identifiable student, and it is stored in the user's own storage rather than on any server operated by the developers.

One caveat must be recorded honestly. The evaluation deployment exposes the backend through a public tunnel without authentication, as noted in Section 8.3. That is acceptable for a supervised demonstration and is not acceptable for real use, and it is the first thing

that would have to change before the system were put in front of students.

Cultural and linguistic considerations. OCR is configured for both English and Bengali, reflecting the reality of instruction in Bangladesh, where lecturers routinely switch languages mid-sentence and write on the board in either. A pipeline that only read English would silently discard part of the lecture, and would do so invisibly, since a missing board block looks exactly like a board that was never written on. This is a recurring theme in the system's failure modes: the dangerous errors are the ones that produce no error, and the design response is to widen coverage rather than to detect absence, because absence in this pipeline is undetectable in principle.

5.3 Ethical Considerations

Ethical questions arise for this system in two distinct registers, and it is worth separating them. The first concerns the conduct of the research itself: what this report claims, and on what evidence. The second concerns the artifact: what a deployed question generator can do to the people who rely on it. The safety and privacy dimensions of the second register are treated in Section 5.2; this section addresses the commitments and obligations that follow from them.

5.3.1 Research Ethics

Two ethical commitments governed this work, and both are visible in the structure of Chapter 4.

No fabricated results. Every number reported in Chapter 4 comes from a measured run, and the report claims nothing beyond what was measured. It would have been easy to present a fuller looking set of results by supplying plausible values for comparisons that were designed but not carried out; that option was rejected, and the chapter reports the comparison that was actually run and analyses it thoroughly instead. The limitations of those measurements, particularly the saturation of the judged dimensions and the weakness of an LLM solver as a proxy for correctness, are stated plainly rather than omitted because they are unflattering. Where a labelling correction had to be applied that cannot be re-derived from the stored artifacts, that fact is disclosed in the results chapter and again in the limitations, rather than quietly incorporated.

No misrepresentation of the system's capability. The tool is an aid to practice, not a substitute for the instructor or the textbook, and this report is explicit about where its output cannot be trusted. A system that generates assessment material carries a specific professional obligation not to overstate its reliability, because the people most likely to over-trust it are precisely the students with the least access to an expert who could correct them: the same students the system is built for.

5.3.2 Ethics of Deployment

Academic integrity. A system that turns a lecture into questions can also be turned toward producing answers, and the boundary between the two is thinner than it looks. LectureAssist is deliberately built as a *self-assessment* instrument: it generates questions for a student to attempt, grades the attempt, and reports which topics were weak. It does not draft submittable coursework, and the short-answer grader returns feedback against the lecture rather than a polished answer to be copied. That boundary is a design decision rather than a technical impossibility, and it is stated here so that it is understood as a commitment which any future extension of the system inherits. An instructor adopting the tool should also be told what it is: an item generator whose output is unreviewed until a human reviews it.

The obligation created by confident error. The safety analysis in Section 5.2 establishes that a fluent, wrong question is the characteristic failure of this class of system. The ethical consequence is that the burden of detection cannot be placed on the learner, because a student who could reliably detect a subtly wrong question about a topic would not need to be tested on it. This is why answer correctness is checked by a second agent that never sees the marked option, why items that fail validation are flagged as overrides rather than silently accepted, and why every generated item carries a timestamp back to the source so a doubt can be resolved against the lecture rather than against the model. These are ethical requirements expressed as architecture, and they are the reason for design choices that would otherwise look like unnecessary cost.

Data, consent and ownership. Lecture recordings contain the voice, and often the image, of an instructor who has not consented to third-party processing, and the material itself is usually the instructor's or the institution's intellectual property. Running entirely on local inference is the strongest available mitigation, because the recording is never transmitted to a provider who could retain it, train on it, or be compelled to disclose it. The legal dimension is treated in Section 5.2; the ethical point is narrower and applies even where the law is silent: a student's right to study a lecture they attended does not extend to redistributing their instructor's teaching, and the system should not make redistribution the path of least resistance. Retention is correspondingly limited to what a session needs.

Bias, coverage and who the system serves. Two kinds of bias bear on this system. The first is inherited: the generator, the judge and the independent solver are all language models trained on corpora that over-represent English and well-resourced academic domains, so question quality will not be uniform across subjects or languages, and this report offers no evidence about performance outside the single English STEM chapter evaluated in Chapter 4. The second is structural and specific to the evaluation: the judge and the solver are drawn from the same model family as the generator, so they share its

blind spots, and agreement between them is evidence of consistency rather than of correctness. That limitation is stated in Chapter 4 as a measurement caveat; it is repeated here because it is also an ethical one. A system that grades itself with a model like itself will systematically fail to detect the errors that model is prone to, and it should not be presented to students as an independent check on their understanding.

Boundaries of responsible use. Taken together, the commitments above define what this system may honestly be claimed to do. It is a practice and revision aid, grounded in a specific lecture, whose output is checked but not guaranteed. It is not an examination instrument, not a substitute for the instructor, and not a source of authority about the subject matter. Deploying it in any of those three roles would exceed what the evaluation in Chapter 4 supports, and the project treats that as an ethical limit rather than a marketing one.

5.4 Impact on Environment and Sustainability

The environmental footprint of an AI system is dominated by the energy consumed during inference at scale. LectureAssist's design happens to be favourable on this axis, largely as a side effect of the constraint that it must run on a free-tier GPU.

Small models, local inference. Question generation runs on a 4B-parameter model rather than a frontier-scale one, and the entire system runs on a single consumer GPU. Inference on a small local model consumes orders of magnitude less energy per request than routing that request to a hyperscale data centre running a model two to three orders of magnitude larger, and it avoids the network transfer and data-centre cooling overhead entirely. No model was trained or fine-tuned during this project, all models are used as released, so the substantial carbon cost of training was not incurred by this work.

Caching and resumability as energy savings. OCR results are cached and keyed by a hash of the video, so re-processing the same lecture costs nothing. Consecutive frames whose extracted text is more than 78% similar are treated as one board state, so a slide left on screen for two minutes contributes one block rather than sixty. Bulk evaluation runs are checkpointed to persistent storage so that a disconnection resumes rather than restarts. Each of these was implemented for practical reasons, free-tier compute is scarce and Colab runtimes die, but each also directly avoids recomputing work that has already been done, which is the cheapest energy saving available and the only one that costs nothing in quality.

The honest counterweight: the agentic loop is not free. The agentic pipeline consumes substantially more compute than a single-shot baseline, because it issues a planning call, up to three drafting calls, a validation call per attempt, and any retrieval calls the

generator elects to make, against the baseline's one. Chapter 4 finds that this buys a real and consistent quality improvement on every metric that discriminates, so the expenditure is not wasted. But "not wasted" is a weaker claim than "efficient", and the report should not pretend otherwise. This is exactly why every ablation and merging table in Chapter 4 carries a mandatory cost column: a configuration that matches the full pipeline's quality using two-thirds of the calls is environmentally as well as economically preferable, and the experiment is designed so that such a configuration would be identified rather than overlooked. Reducing the call count at fixed quality is simultaneously a performance goal, a cost goal and an environmental one, and the three do not conflict.

Sustainability of access. A tool that requires a subscription is sustainable only for as long as the student can pay for it, and only for as long as the vendor chooses to keep that model available at a stable price. Models are deprecated, prices change, and free tiers are withdrawn. Because LectureAssist depends on no external service, it will continue to work on hardware a student already owns, for as long as they own it, with weights they have already downloaded. This is an important form of sustainability that is easy to overlook when sustainability is framed purely in terms of energy consumption: a system that stops working when a company changes its pricing has a lifespan measured in business decisions rather than in hardware.

Chapter 6

Project Planning and Budget

This chapter documents how the project was planned and resourced. It sets out the development methodology, decomposes the work into a structured breakdown, presents the thirty-week schedule as a Gantt chart with explicit milestones, assigns responsibility across the team, records the risks that were identified and how they were mitigated, and closes with the project budget. A recurring theme throughout is that the schedule and the budget were shaped by the same constraint that shaped the architecture: the system had to be buildable and runnable on free-tier hardware with no paid inference service.

6.1 Development Methodology

The project followed an *incremental, artifact-driven* methodology rather than a strict waterfall or a fixed-sprint agile process. This choice was forced by the nature of the system. LectureAssist is a pipeline in which each stage consumes the output of the previous one, and several of those stages, optical character recognition, speech transcription, bulk question generation, are expensive enough that they cannot be re-run casually on a free-tier GPU. The pipeline was therefore built stage by stage, and each stage was required to emit a *durable, inspectable artifact* to persistent storage before the next stage was started.

Three consequences followed from that decision, and all three shaped the schedule visible in Section 6.3.

Stages could be developed and debugged independently. Because the OCR stage writes a cached, hashed transcript of every board state to disk, the retrieval stage could be developed against that cached file without re-running OCR even once. This is why the extraction tasks and the retrieval task overlap in the schedule rather than running strictly in series.

Expensive work became resumable. A Colab runtime disconnection is not an exceptional event on the free tier; it is a routine one. Checkpointing every generation and judging run to Google Drive converted a class of failure that would otherwise have cost

days of re-computation into one that costs minutes. Without this, the bulk evaluation task in weeks 24–27 would not have been feasible at all inside the schedule.

Measurement was decoupled from generation. Generation and scoring were deliberately implemented as separate programs communicating through a JSON artifact on disk. This meant the evaluation harnesses could be written, corrected and re-run against an already-generated corpus without regenerating a single question, which is precisely what happened when the first EQGBench protocol implementation was judged unfaithful to the published prompt and had to be rewritten.

6.2 Work Breakdown Structure

The project was decomposed into five phases containing twenty work packages. Phases correspond to coherent subsystems rather than to calendar periods, which is why they overlap in time. Table 6.1 lists them with their deliverables.

Table 6.1: Work breakdown structure. Each work package produces a deliverable that a later package consumes.

WP	Work package	Deliverable	Weeks
Phase 1, Planning and Research			
1.1	Literature review	Annotated bibliography; research gap	1–5
1.2	Requirement analysis and scope	Functional requirement list	2–4
1.3	Toolchain selection, environment setup	Working Colab + Ollama runtime	4–6
Phase 2, Multimodal Extraction			
2.1	Frame sampling and scene classification	Four-class frame router	5–8
2.2	Scene-adaptive OCR	Deduplicated board-text blocks	7–11
2.3	Speech transcription	Timestamped transcript segments	8–10
2.4	Timeline fusion and document parsing	Unified lecture timeline	10–12
2.5	Retrieval index construction	FAISS index; `search_lecture` tool	11–13
Phase 3, Agentic Generation Pipeline			
3.1	Agent framework and orchestrator	`BaseAgent`; agent event log	12–15
3.2	Planner and Generator agents	Topic–difficulty slots; drafted items	14–17
3.3	Validator and Refiner loop	Verdict plus remediation action	16–19
3.4	Five question types; multi-tier grading	Grading cascade; feedback	18–21
3.5	Adaptive difficulty engine	Persistent per-topic profile	20–22
Phase 4, Interface and Integration			
4.1	Flask API and tunnel deployment	Public JSON API	15–18
4.2	Web interface and live agent workspace	Single-page front end	17–22
4.3	End-to-end integration and debugging	Demonstrable system	21–24
Phase 5, Evaluation and Documentation			
5.1	Evaluation harnesses	Three scoring programs	22–26
5.2	Bulk generation and judging runs	Scored question corpus	24–27
5.3	Ablation and merging experiments	Per-agent attribution	26–28
5.4	Report, poster and defence	This report; A0 poster	25–30

6.3 Project Schedule

Figure 6.1 presents the thirty-week schedule. The project spans two academic terms: CSE 499A covers weeks 1–15 and CSE 499B covers weeks 16–30, marked on the chart by the vertical divider. The division is not merely administrative. By the end of week 15 the system had to be able to take a lecture video and produce a grounded, retrievable representation of it; everything that generates, validates or evaluates questions depends on that representation existing and was therefore scheduled after it.

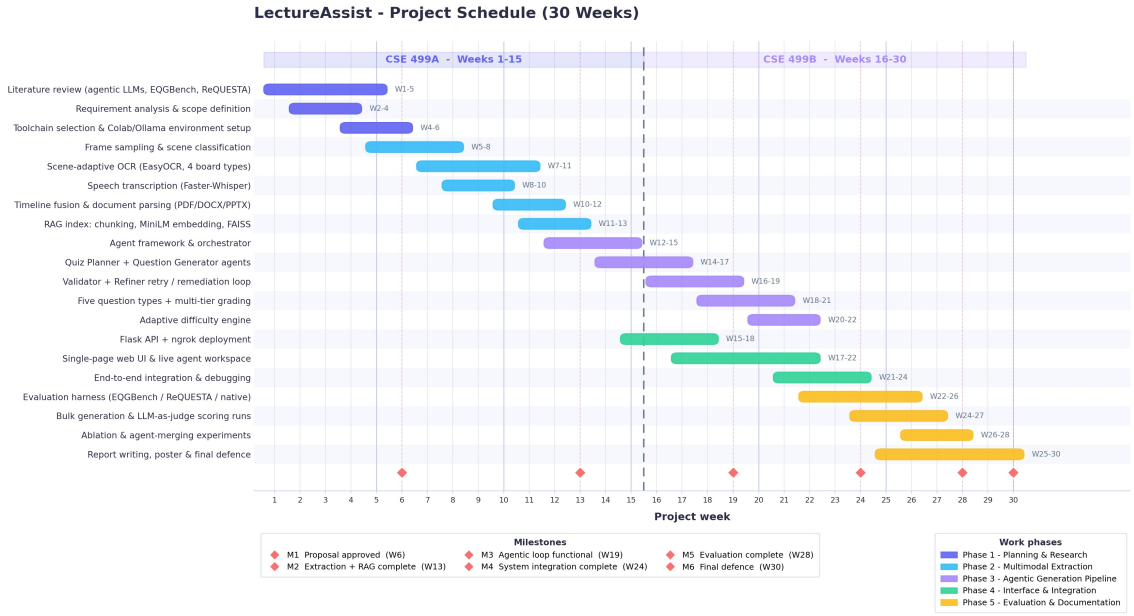

Figure 6.1: Project schedule across thirty weeks, showing twenty work packages grouped into five phases, the CSE 499A/499B term boundary at week 15, and six milestones. Overlapping bars reflect the artifact-driven methodology of Section 6.1: a downstream stage begins as soon as the upstream stage emits a usable artifact, not when it is finished.

Three scheduling decisions in that chart are worth justifying explicitly, because each represents a deliberate trade-off rather than a default.

Extraction and retrieval overlap. Work package 2.5 begins in week 11, while OCR work package 2.2 is still running. This was possible only because of the caching decision described above, and it bought roughly two weeks of slack that were later consumed by the evaluation phase.

The interface was built early and in parallel. Front-end work begins in week 17, well before the pipeline was complete. The reasoning was that the live agent workspace, which visualises the per-question Plan, Generate, Validate and Retry trace as it happens, is not merely a presentation layer but a debugging instrument. Having it available during weeks 18–21 materially accelerated the development of the validator and refiner, because a failing remediation decision could be seen directly rather than reconstructed from logs.

Evaluation was given eight weeks, and needed them. Weeks 22–30 are dominated by measurement rather than construction. This allocation was made after the first pilot judging run showed that a faithful implementation of a published protocol is considerably more work than it appears: EQGBench requires five separate judging calls per question, repeated over three independent rounds, and ReQUESTA turned out to publish no judge prompt at all. Both harnesses had to be written from the papers themselves rather than adapted from released code.

Table 6.2: Project milestones and their acceptance criteria.

ID	Week	Milestone	Acceptance criterion
M1	6	Proposal approved	Scope, research gap and toolchain agreed with the supervisor
M2	13	Extraction and retrieval complete	A lecture video yields a fused, timestamped, retrievable timeline
M3	19	Agentic loop functional	A question is planned, drafted, validated and remediated end to end
M4	24	System integration complete	All five question types, grading and adaptation reachable from the interface
M5	28	Evaluation complete	Bulk corpus generated and scored; metrics computed from a real run
M6	30	Final defence	Report submitted, poster printed, system demonstrated live

6.4 Team Roles and Responsibilities

The team consists of three members working under one supervisor. Responsibility was assigned per work package using a RACI allocation, Responsible, Accountable, Consulted, Informed, reproduced in Table 6.3. In every row exactly one member is Accountable, so that no work package could be left un-owned.

Table 6.3: RACI responsibility matrix. R = Responsible (does the work), A = Accountable (owns the outcome), C = Consulted, I = Informed.

Work package	Iftikhar	Salman	Jarinah	Supervisor
Literature review and research gap	R	C	A	C
Requirement analysis and scope	C	A	R	A
Extraction: OCR and scene classification	A	R	C	I
Extraction: speech transcription	R	A	C	I
Timeline fusion and document parsing	A	C	R	I
Retrieval index and search tool	C	A	R	I
Agent framework and orchestrator	R	A	C	C
Planner, Generator, Validator, Refiner	C	A	R	C
Question types and grading cascade	A	R	C	I
Adaptive difficulty engine	R	C	A	I
Flask API and deployment	A	R	I	I
Web interface and agent workspace	R	I	A	I
Evaluation harnesses	C	A	R	C
Bulk runs and result analysis	R	A	C	C
Report, poster and defence	R	R	R	A

6.5 Risk Management

Table 6.4 is the risk register maintained across the project. It is reproduced here as it was used rather than as a retrospective idealisation, which is why it includes risks that materialised. Likelihood and impact are rated on a three-point scale (L/M/H).

Table 6.4: Risk register. Risks marked *occurred* were realised during the project and were handled by the stated mitigation.

Risk	**L**	**I**	**Mitigation**	**Status**
Colab runtime disconnects mid-run	H	H	Checkpoint every artifact to Drive after each question; make all long runs resumable rather than restartable	occurred
Free-tier GPU memory insufficient for both the LLM and the embedding model	M	H	Pin generation to the 4B model; use a 384-dimension embedding model that co-exists with it; keep the 12B model for judging only	occurred
The 4B generator returns malformed JSON	H	M	A staged JSON repair chain (escape fixing, quote normalisation, block slicing) followed by retry on the same model; never silently escalate to the 12B	occurred
Judge API rate limiting stalls the evaluation	M	H	Row-level checkpointing so a judging run resumes; smoke-test with a small sample before committing to a full run	occurred
Published protocol cannot be faithfully reproduced	M	M	Transcribe rubrics directly from the papers; declare every deviation explicitly rather than silently approximating	occurred
OCR fails on handwritten or low-contrast board work	H	M	Classify each frame before preprocessing; per-class contrast enhancement and polarity inversion; accept that some loss is unavoidable and state it	occurred
Team member unavailable during a critical week	M	M	RACI matrix ensures every package has a named backup in the Responsible column	not realised
Scope creep into features not required by the research question	M	M	Features outside the Plan–Generate–Validate–Refine loop were deferred unless they served evaluation or debugging	contained

6.6 Budget

Figure 6.2 presents the project budget. The total is 8,200 BDT, approximately USD 67 at the conversion rate stated in the figure.

LectureAssist - Project Budget

30-week senior design project - all inference runs locally on free-tier hardware; no paid API is used anywhere in the generation path.

CATEGORY	ITEM	JUSTIFICATION	COST (BDT)
Compute	Google Colab free tier - NVIDIA T4 GPU (16 GB)	*All model inference; free tier is the target deployment*	0 (free)
Compute	Google Drive (15 GB, free tier)	*Session persistence across Colab disconnects*	0 (free)
Models	Gemma 3 (4B / 12B) open weights via Ollama	*Generation, summarisation, grading - no API key*	0 (free)
Software	EasyOCR, Faster-Whisper, FAISS, Sentence-Transformers	*Extraction, transcription and retrieval stack (open source)*	0 (free)
Software	Flask, PyMuPDF, OpenCV, SymPy	*Backend API, document parsing, answer-key verification*	0 (free)
Network	ngrok free tier tunnel	*Exposes the Colab backend to the browser front end*	0 (free)
Evaluation	DeepSeek-R1 API credits (EQGBench judging protocol)	*Independent judge required by the published protocol*	1,200
Hardware	Development laptops (3 x existing, no purchase)	*Front-end development and report writing*	0 (free)
Operations	Internet connectivity (3 members x 30 weeks)	*Colab sessions, model pulls, collaboration*	2,000
Operations	A0 poster printing and report binding	*Final presentation and submission deliverables*	4,000
Reserve	Contingency reserve	*Extra judging credits, reprints*	1,000
	TOTAL PROJECT COST		**8,200 BDT (approx. USD 67)**

Cost avoided by the local-first design

Every question in the study was generated and graded by open-weight models on the free T4. Only the independent judge required by the EQGBench protocol was paid for, which is why the single non-zero compute line above is an evaluation cost, not a generation cost.

Currency: Bangladeshi Taka (BDT). Conversion at 1 USD = 122 BDT. Hardware owned by the team before the project began is listed at zero cost.

Figure 6.2: Project budget. Every component on the generation path is free: the compute is a free-tier NVIDIA T4, the models are open weights served locally, and the extraction, retrieval and serving stack is open source. The single non-zero technical line is the independent judge required by the EQGBench protocol, which is an evaluation cost rather than a system cost.

The structure of that table is the point of it, more than the total. Three observations follow.

The system itself costs nothing to run. Every line associated with producing study material, compute, models, extraction stack, retrieval, serving, is zero. This is a direct consequence of the constraint stated in Chapter 5: a tool whose purpose is to make high-volume retrieval practice available to students who cannot afford per-token cloud pricing cannot itself depend on per-token cloud pricing. The engineering cost of that decision is real and is paid elsewhere in this report: it is why the generator is a 4B model, and why so much of Chapter 3 is concerned with extracting reliable structured output from a small model.

The one paid line is a measurement cost, not a product cost. EQGBench specifies DeepSeek-R1 as its judge, and a faithful reproduction of a published protocol requires using the model the protocol specifies. Substituting a locally served model would have made the numbers incomparable with the published figures, which is the entire reason for implementing the protocol faithfully in the first place. A user of LectureAssist never incurs this cost; only a party reproducing the evaluation does.

The dominant real cost is human time, which does not appear in the table. Roughly 2,000 BDT of the total is connectivity and 4,000 BDT is printing and binding, so the budget is dominated by overheads that any project of this length would incur regardless of its subject. The genuine investment in this project was the thirty weeks of work documented in Section 6.3, and the fact that a system of this scope could be built and evaluated for well under USD 100 of direct cost is itself a result worth recording, since it establishes that the approach is reproducible by any student team with a laptop and an internet connection.

Cost avoided. It is worth stating what the alternative would have been. The evaluation reported in Chapter 4 involved generating hundreds of questions per pipeline configuration, each of which consumed multiple LLM calls under the agentic loop, and then judging every resulting question. Executed against a commercial frontier API at prevailing per-token rates, that workload would have cost one to two orders of magnitude more than the entire budget above, and, critically, it would have made the ablation and merging experiments of Section 4.3 unaffordable, since those multiply the corpus by the number of configurations under test. Running locally did not merely save money; it made a class of experiment possible that would otherwise have been out of reach.

Chapter 7

Complex Engineering Problems and Activities

This chapter maps LectureAssist onto the Complex Engineering Problem (CEP) and Complex Engineering Activity (CEA) attributes, showing where the project required depth of knowledge, resolution of conflicting requirements, and analysis beyond the routine application of a known method. Each attribute is addressed with reference to the specific decision, defect or measurement in this report that exercises it, rather than by general assertion.

7.1 Complex Engineering Problems (CEP)

Table 7.1: Complex Engineering Problem attributes addressed by LectureAssist.

Attr.	Description	How the project addresses it
P1	Depth of knowledge required (K3–K8)	The system spans computer vision and image preprocessing for scene-adaptive OCR (K3–K4), automatic speech recognition, transformer language models, embeddings and vector retrieval (K4–K5), multi-agent software architecture and API/system design (K5–K6), statistical evaluation design, LLM-as-judge, F_1 agreement between the internal validator and an independent judge, calibration against a known-answer set (K5, K8), and the research literature on question generation, chain-of-thought and program-of-thought prompting, and agentic architectures (K8).

Attr.	Description	How the project addresses it
P2	Range of conflicting requirements	Several requirements pull directly against each other. *Question quality vs. cost*: the agentic loop produces better-grounded, less repetitive and more answer-correct questions but consumes roughly an order of magnitude more LLM calls than a single-shot request. *Model size vs. VRAM*: a larger generator would produce better questions, but the generator, judge, Whisper, OCR and embedding models must all coexist on one 16 GB T4. *Yield vs. scientific honesty*: allowing the 4B generator to escalate to the 12B model on malformed output would have raised yield, but would have falsified the claim that the questions were written by the 4B, so escalation was disabled at a known cost in yield. *Accept rate vs. diversity*: a pipeline can maximise its accept rate by repeatedly producing one good question, which forced the adoption of a duplicate-penalised effective acceptance metric (Eq. 4.1) in place of raw acceptance.
P3	Depth of analysis required	There is no unique correct design, and no off-the-shelf metric for "is this a good generated question". Selecting a solution required analysing four generation strategies (agentic, plain, CoT, PoT) against each other under a metric suite that had itself to be designed, and it required identifying and eliminating confounds, unequal content coverage between pipelines, a truncated summary that made whole sections of the chapter unreachable by the planner, and silent model escalation, any one of which would have invalidated the comparison. The central analytical finding is that the four judged quality dimensions *saturate* and therefore discriminate nothing, so that the entire result rests on corpus-level automated measures that a conventional quality-only evaluation would never have computed. That conclusion is invisible without the analysis and inverts the naive reading of the data.
P4	Familiarity of issues	The project integrates infrequently encountered components: EasyOCR with scene-dependent preprocessing, Faster-Whisper, Gemma 3 served through Ollama, Sentence-BERT embeddings, a FAISS index, an LLM-as-judge evaluation harness, SymPy for machine-verified answer keys, and a Flask/ngrok deployment onto ephemeral free-tier GPU infrastructure.
P5	Extent of applicable codes	No standard or code of practice exists for automatic educational question generation or for the evaluation of LLM-generated assessment items. The evaluation protocol, what to measure, how to verify an answer, how to calibrate the judge, had to be designed from the research literature rather than taken from a standard. Where published protocols did exist they had to be reconstructed from the papers themselves: EQGBench has no public code release, and ReQUESTA publishes no judge prompt at all because its rubric was applied by human raters.
P6	Extent of stakeholder involvement	Stakeholders include students (the primary users), instructors (whose lectures are the input and whose assessments the tool must not undermine), institutions (which hold copyright in lecture material and are accountable for academic integrity), and the open-model community whose licences govern the deployed models. Their requirements conflict: a student wants unlimited questions immediately, while an instructor requires that the tool not present unverified answers as authoritative, and an institution requires that copyrighted lecture material not be transmitted to a third-party processor.

Attr.	Description	How the project addresses it
P7	Interdependence	The subsystems are tightly coupled: OCR and ASR quality bound the summary; the summary bounds the planner's topic choice; the planner bounds question diversity; the retrieval index bounds grounding; and the judge and Answer Generator bound the credibility of every reported result. Two of the defects found during this project, a truncated summary and a capped content window, were failures of exactly this interdependence: a bug several stages upstream degraded question diversity many stages downstream, with no local symptom at the point of failure. The pipeline labelling error disclosed in Section 4.4.1 is a third instance, in which a fault in the recording of provenance propagated into the interpretation of every downstream measurement.

Table 7.1 shows that the project engages all seven CEP attributes, with P2 (conflicting requirements) and P3 (depth of analysis) being the most heavily exercised. It is worth noting that P2 and P3 are linked in this project rather than independent: almost every conflicting requirement listed under P2 was resolved by *measuring* the trade-off rather than by asserting a preference, and the instruments built to perform those measurements are what generated the depth of analysis recorded under P3.

7.2 Complex Engineering Activities (CEA)

Table 7.2: Complex Engineering Activity attributes addressed by LectureAssist.

Attr.	Description	How the project addresses it
A1	Range of resources	The project draws on human resources (a three-member team with a supervisor), modern computational tools (a GPU runtime, six distinct deep-learning models, a vector index, a symbolic algebra system), open-licensed source material, and cloud compute, managed under a hard constraint of zero paid API spend on the generation path, as documented in the budget of Section 6.6.
A2	Level of interactions	The work required interaction between the extraction, retrieval, generation, evaluation and interface subsystems, each with its own failure modes, as well as between team members under the responsibility allocation of Table 6.3, the project supervisor (whose requirements shaped the evaluation design, notably the demands for CoT and PoT baselines and for treating question generation and answer generation as separate cross-checked agents), and the end users whose trust in the output determines whether the system is safe to use.

Attr.	Description	How the project addresses it
A3	Innovation	The project introduces three novel elements: scene-adaptive OCR that classifies each frame before choosing its preprocessing, so that a lecture's *board*: not merely its audio, becomes usable content; a validator agent that selects *how* to remediate a rejected question (regenerate, fetch more context, or replan the topic) rather than merely rejecting it; and an evaluation protocol that separates question quality from answer correctness by having a distinct Answer Generator agent re-solve every question blind, with the solver itself calibrated on a machine-verified answer key.
A4	Consequences to society and the environment	The system widens access to effective retrieval practice for students who cannot pay for cloud AI services, supporting UN SDG 4 (Quality Education) and SDG 10 (Reduced Inequalities). Running small models locally is substantially less energy-intensive per request than hyperscale cloud inference, and no model was trained. The principal risk, a fluent question with a wrong marked answer, is analysed and bounded rather than dismissed, and the report states explicitly that generated questions are suitable for practice and not for graded assessment without human review. Chapter 5 develops these consequences in full.
A5	Familiarity	The project requires familiarity with computer vision, speech recognition, large language models and prompting strategies, retrieval-augmented generation, agentic architectures, experimental design and statistical evaluation, web API development, and deployment onto constrained, ephemeral GPU infrastructure.

Table 7.2 demonstrates that the project satisfies all five Complex Engineering Activity attributes.

7.3 Mapping to Knowledge Profiles

Table 7.3 records which Washington Accord knowledge profiles the work draws on and where in this report the evidence for each can be found, so that the mapping can be checked against the substance of the project rather than accepted on assertion.

Table 7.3: Knowledge profile coverage and where each is evidenced in this report.

Profile	Description	Evidence in this report
K3	Engineering fundamentals	Image preprocessing, contrast-limited adaptive histogram equalisation, thresholding, polarity inversion, selected per frame class (Section 3.1.2)
K4	Engineering specialist knowledge	Transformer language models, speech recognition, sentence embeddings and approximate nearest-neighbour search (Chapter 2)
K5	Engineering design	The five-agent Plan–Retrieve–Generate–Validate–Refine architecture and its alternatives (Section 3.3.1, Figure 3.2)
K6	Engineering practice	Component selection against stated alternatives and rationale (Table 3.1); deployment on constrained infrastructure

Profile	Description	Evidence in this report
K7	Comprehension of the role of engineering in society	Access, safety, privacy and environmental analysis (Chapter 5)
K8	Research literature	Two published protocols reconstructed from their source papers and applied with declared deviations (Section 4.2)

Chapter 8

Conclusion

This chapter summarises what was built and what was learned, states the limitations of the work plainly, and sets out the improvements that follow most directly from them.

8.1 Summary

LectureAssist is an end-to-end, fully local agentic system that turns a raw lecture video or academic document into validated, difficulty-controlled study and assessment material. It fuses two extraction channels, scene-adaptive optical character recognition that classifies each frame before preprocessing it, so that slides, blackboards, whiteboards and on-screen text are each handled correctly, and Faster-Whisper speech transcription [35], into a single timestamped lecture timeline. That timeline is summarised, chunked, embedded with Sentence-BERT [24] and indexed in FAISS [26], and is then consumed by a multi-agent pipeline that plans questions across topics, retrieves its own evidence when it judges its context insufficient, generates one question at a time, validates each against difficulty, grounding and instruction compliance, and, on failure, chooses *how* to remediate before retrying. Around this sit a retrieval-grounded chat interface, a multi-tier grader that falls back from exact matching through fuzzy similarity to rubric evaluation by a larger model, and an adaptive engine that steers subsequent quizzes toward the topics a student is weakest on. Everything runs on open-weight Gemma 3 models [36] served locally through Ollama [37] on a single free-tier GPU, with no paid API anywhere in the system.

The project has a second contribution that is methodological rather than architectural. It takes seriously a question the field usually does not ask: not *does the generator write good questions*, but *does it mark the right answer*. Question generation and answer generation are treated as two distinct agents, and their outputs are cross-checked by having a larger model re-solve every generated question without ever seeing the marked option. This is deliberately harder to pass than a fluency-based evaluation, and it is the only part of the evaluation that can detect the failure mode identified in Chapter 1: a question that reads perfectly and is wrong.

8.2 Findings

Three findings come out of the evaluation in Chapter 4, and they are of quite different kinds.

The agentic pipeline beats single-shot prompting on every metric that discriminates. Measured against a plain single-pass baseline on a college calculus chapter at n=200 questions per pipeline, the agentic loop produces fewer near-duplicate questions (0.280 against 0.385), is better grounded in the source text (0.624 against 0.557), achieves a higher effective acceptance rate (0.720 against 0.612), and agrees more often with an independent solver (0.970 against 0.940). The result holds at all three target difficulties without a single reversal, and the margin widens as difficulty rises, on hard questions the agentic pipeline roughly halves the duplicate rate. Four independent measures, three difficulty strata, one direction: that consistency is what makes the finding credible.

The most instructive finding is that the obvious metrics do not work. All four judged quality dimensions saturate. Every value falls between 4.36 and 5.00 on a five-point scale and correctness reaches the ceiling exactly, so the apparent agentic wins on those rows, 4.93 against 4.86, 5.00 against 4.98, carry almost no information. A metric that cannot go up cannot discriminate. This is not a peculiarity of our judge; the same collapse of judged quality scales is reported across the educational question-generation literature reviewed in Chapter 2, for every strong model tested. The practical consequence is significant and generalises beyond this project: **an evaluation of automatic question generation that reports only judged quality scores will conclude that every reasonable system is excellent and that none is distinguishable from any other.** The signal in this study lives entirely in the automated corpus-level measures, duplication, grounding, effective acceptance and independent answer match, and a study that had not computed them would have found nothing at all.

Winning as a whole is not the same as knowing why. The measurement establishes that the five-agent pipeline outperforms one LLM call. It does not establish which of the five agents is responsible, and the pipeline costs roughly an order of magnitude more compute than the call it beats. The mechanism proposed in Section 4.4.1: that validation and reasoned retry suppress duplication and pull generation back toward the source, is consistent with every number in the study and is not established by any of them. It is labelled a hypothesis throughout this report for that reason, and the leave-one-out and merging families of Section 4.3 exist specifically to test it.

8.3 Limitations

The pipeline labelling rests on generation records, not on the artifacts. As disclosed in Section 4.4.1, the two arms were interchanged at generation time and the

correction was applied afterwards. The script that assigned the original labels is no longer in the repository, so the assignment cannot be re-derived from the stored corpus, whose JSON keys still carry the uncorrected labels. Every headline result in this report depends on that correction being right. This is the single most consequential limitation of the work and it is placed first for that reason; the remedy is a confirming re-run, and it is the first item in Section 8.4.

The judge and the Answer Generator are language models, not humans. All quality scores come from `gemma3:12b` rather than from human raters. LLM judges are known to reward fluency and verbosity [38], and this judge shares a model family with the generator it grades, so it plausibly shares its blind spots. Absolute scores should be read as a relative signal between pipelines, never as an objective measure of pedagogical quality.

Generator–solver agreement is not the same as correctness. The independent-match rate measures how often two models agree, and two models drawn from the same family agree partly because they fail in the same way. If generator and solver share a misconception they agree and are both wrong, and the metric records a success. The calibration set exists precisely to bound this, both models solve items whose answers are known independently, with the computational items machine- computed using SymPy [31] rather than typed by hand, but until that calibration is reported the answer-correctness figures establish a *ranking* between pipelines rather than a trustworthy absolute correctness rate.

The result is a comparison of two pipelines, not an attribution. The evaluation establishes that the full agentic loop outperforms a single-shot baseline. It does not identify which of the five agents produces that advantage, because a configuration that wins as a whole cannot be decomposed by observing it. The ablation and merging designs of Section 4.3 are specified precisely to close that gap, and until they are carried out the attribution of the advantage to any particular agent stays an inference drawn from the mechanism rather than a measured result.

Single domain, single document. All results come from one chapter of one subject: limits, from OpenStax *Calculus, Volume 1* [39]. Mathematics is unusually favourable to program-of-thought prompting [40] and unusually hostile to OCR, and nothing here establishes that these findings transfer to a humanities lecture, a lab-based science course, or material whose structure is narrative rather than definitional.

The extraction layer bounds everything above it. OCR on handwritten board work is imperfect, transcription degrades with accent and background noise, and mathematics rendered as an image inside a PDF is not extracted at all. Every defect in extraction

propagates silently upward, and silence is the problem: a missing board block is indistinguishable from a board that was never written on, so the question generator cannot know that material is absent and cannot flag it.

Deployment is not production-grade. The system runs on a free Colab runtime behind an ngrok tunnel, with a single-user interface, no authentication, and session state on Google Drive. This is adequate to demonstrate and evaluate the system; it is not a deployable service, and the security posture in particular, an unauthenticated public tunnel to a machine holding uploaded course material, would need to be addressed before any real use.

8.4 Future Improvement

Confirm the arm labelling before anything else. A single re-run of the two pipelines with the labelling recorded end to end, in the artifact itself rather than in a separate script, would settle the question on which every result in this report depends. It is cheap relative to everything else proposed here and it should be done first. The lesson generalises: provenance should be written into the artifact at generation time, not reconstructed afterwards.

Run the ablations, and read them against cost. The leave-one-out and merging families are designed, specified and in two cases already implemented. They are what turns "the pipeline wins" into "this agent is why, and it costs this much". The − validator and − refiner pair is the sharpest of them, because it separates *detecting* a bad question from *acting* on that detection, and if detection alone captures most of the benefit, the refiner call can be removed outright for a substantial saving.

Evaluate under a published benchmark. Every result in this report comes from metrics defined by this project. That is enough to rank two pipelines against each other, but it cannot say how the system compares with work published elsewhere, because nobody else reports these numbers. Scoring the existing corpus under one of the standard educational question-generation protocols surveyed in Chapter 2 would place this system beside published figures on a published instrument, which no amount of project-native metrics can substitute for. Two such harnesses were written during this project and are available to be run against the existing corpus.

Strengthen answer verification. The weakest link in the evaluation is the solver. Verifying computational answers symbolically with SymPy, as is already done when constructing the calibration set, would replace a model opinion with a machine-checked fact for exactly the question type where a generator is most likely to be wrong. Using a solver from a different model family, or an ensemble, would break the shared-blind-spot problem that currently inflates the agreement rate.

Human evaluation. A modest human study: a few instructors rating a sample of generated questions and independently answering them, would anchor the LLM judge to a ground truth and quantify how far its scores can be trusted. Given that the judged dimensions saturate, the most valuable design would not ask instructors to score quality on a scale at all, but to make forced-choice comparisons between paired questions from the two pipelines, which does not suffer from ceiling effects.

Broaden the domain. Repeating the comparison on a non-mathematical subject would test whether the agentic advantage is general or is an artifact of a domain in which answers are computed and therefore easy to get wrong. A humanities lecture, where grounding is harder to measure and correctness is less binary, is the sharpest available test.

Close the loop on diversity. Duplicate rate is the metric on which the agentic pipeline earns its advantage, and it is still far from zero: better than one question in four is a near-duplicate of another. Two mechanisms follow directly from the diagnosis, maintaining a coverage map of which regions of the document have already been used and penalising the planner for returning to them, and moving the duplicate check inside the validation loop so that a repetitive question is rejected at generation time rather than discovered at evaluation time. The second is a small change with a measurable payoff and is the natural next engineering step.

References

[1] B. S. Bloom, *Taxonomy of Educational Objectives: The Classification of Educational Goals. Handbook I: Cognitive Domain.* David McKay Company, 1956.

[2] L. W. Anderson and D. R. Krathwohl, *A Taxonomy for Learning, Teaching, and Assessing: A Revision of Blooms Taxonomy of Educational Objectives.* Longman, 2001.

[3] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, "A systematic review of automatic question generation for educational purposes," *International Journal of Artificial Intelligence in Education*, vol. 30, no. 1, pp. 121–204, 2020.

[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017, pp. 5998–6008.

[5] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proceedings of NAACL-HLT*, 2019, pp. 4171–4186.

[6] C. Zhou, J. Wang, K. Li, P. Yang, and W. Sun, "From answers to questions: EQG-Bench for evaluating large language models educational question generation," *arXiv preprint arXiv:2508.10005*, 2025.

[7] Y. Tian, H. Zhang, L. Wang, M. Glass, and S. Bansal, "Cognitively diverse multiple-choice question generation: A hybrid multi-agent framework with large language models," *arXiv preprint arXiv:2602.03704*, 2026.

[8] M. Heilman and N. A. Smith, "Good question! statistical ranking for question generation," in *Proceedings of NAACL-HLT*, 2010, pp. 609–617.

[9] X. Du, J. Shao, and C. Cardie, "Learning to ask: Neural question generation for reading comprehension," in *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2017, pp. 1342–1352.

[10] Q. Zhou, N. Yang, F. Wei, C. Tan, H. Bao, and M. Zhou, "Neural question generation from text: A preliminary study," in *National CCF Conference on Natural Language Processing and Chinese Computing (NLPCC)*, 2017, pp. 662–671.

[11] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 24 824–24 837.

[12] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 22 199–22 213.

[13] L. Ouyang, J. Wu, X. Jiang, D. Almeida, C. L. Wainwright, and P. Mishkin, "Training language models to follow instructions with human feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 27 730–27 744.

[14] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, and S. Wiegreffe, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[15] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[16] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, and E. Zhu, "AutoGen: Enabling next-generation large language model applications via multi-agent conversation," *arXiv preprint arXiv:2308.08155*, 2023.

[17] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, and C. Zhang, "MetaGPT: Meta programming for a multi-agent collaborative framework," in *International Conference on Learning Representations (ICLR)*, 2024.

[18] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "BLEU: A method for automatic evaluation of machine translation," in *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2002, pp. 311–318.

[19] C.-Y. Lin, "ROUGE: A package for automatic evaluation of summaries," in *Text Summarization Branches Out: Proceedings of the ACL-04 Workshop*, 2004, pp. 74–81.

[20] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu, "G-Eval: Natural language generation evaluation using GPT-4 with better human alignment," in *Proceedings of EMNLP*, 2023, pp. 2511–2522.

[21] JaidedAI, "EasyOCR: Ready-to-use optical character recognition with eighty supported languages," https://github.com/JaidedAI/EasyOCR, 2020.

[22] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, and N. Goyal, "Retrieval-augmented generation for knowledge-intensive natural language processing tasks," in

Advances in Neural Information Processing Systems (NeurIPS), vol. 33, 2020, pp. 9459–9474.

[23] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, and H. Wang, "Retrieval-augmented generation for large language models: A survey," *arXiv preprint arXiv:2312.10997*, 2023.

[24] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using siamese BERT-networks," in *Proceedings of EMNLP-IJCNLP*, 2019, pp. 3982–3992.

[25] W. Wang, F. Wei, L. Dong, H. Bao, N. Yang, and M. Zhou, "MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 5776–5788.

[26] J. Johnson, M. Douze, and H. Jegou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

[27] Pallets Projects, "Flask: A lightweight WSGI web application framework," https://flask.palletsprojects.com, 2010.

[28] ngrok Inc., "ngrok: Unified ingress platform," https://ngrok.com, 2013.

[29] Google Research, "Google Colaboratory," https://colab.research.google.com, 2017.

[30] Artifex Software, "PyMuPDF: Python bindings for the MuPDF rendering library," https://pymupdf.readthedocs.io, 2016.

[31] A. Meurer, C. P. Smith, M. Paprocki, O. Certik, S. B. Kirpichev, and M. Rocklin, "SymPy: Symbolic computing in Python," *PeerJ Computer Science*, vol. 3, p. e103, 2017.

[32] H. L. Roediger and J. D. Karpicke, "Test-enhanced learning: Taking memory tests improves long-term retention," *Psychological Science*, vol. 17, no. 3, pp. 249–255, 2006.

[33] J. D. Karpicke and J. R. Blunt, "Retrieval practice produces more learning than elaborative studying with concept mapping," *Science*, vol. 331, no. 6018, pp. 772–775, 2011.

[34] J. Dunlosky, K. A. Rawson, E. J. Marsh, M. J. Nathan, and D. T. Willingham, "Improving student learning with effective learning techniques: Promising directions from cognitive and educational psychology," *Psychological Science in the Public Interest*, vol. 14, no. 1, pp. 4–58, 2013.

[35] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023, pp. 28 492–28 518.

[36] Gemma Team, Google DeepMind, "Gemma 3 technical report," *arXiv preprint arXiv:2503.19786*, 2025.

[37] Ollama, "Ollama: Get up and running with large language models locally," https://ollama.com, 2023.

[38] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, and Y. Zhuang, "Judging LLM-as-a-judge with MT-Bench and Chatbot Arena," in *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, vol. 36, 2023.

[39] G. Strang and E. Herman, *Calculus, Volume 1.* OpenStax, Rice University, 2016.

[40] W. Chen, X. Ma, X. Wang, and W. W. Cohen, "Program of thoughts prompting: Disentangling computation from reasoning for numerical reasoning tasks," *Transactions on Machine Learning Research*, 2023.